Neural Networks 112 (2019) 54–72

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

A stochastic variational framework for Recurrent Gaussian Processes models

César Lincoln C. Mattos *, Guilherme A. Barreto

Computer Science Department (DC), Federal University of Ceará (UFC), Center of Sciences, Campus of Pici, Fortaleza, Ceará, Brazil
Department of Teleinformatics Engineering (DETI), Federal University of Ceará (UFC), Center of Technology, Campus of Pici, Fortaleza, Ceará, Brazil

ARTICLE INFO

Article history:
Received 10 April 2018
Received in revised form 31 October 2018
Accepted 20 January 2019
Available online 1 February 2019

Keywords:
Gaussian Processes
Dynamical modeling
Variational inference
Stochastic learning

ABSTRACT

Gaussian Processes (GPs) models have been successfully applied to the problem of learning from sequential observations. In such context, the family of Recurrent Gaussian Processes (RGPs) have been recently introduced with a specifically designed structure to handle dynamical data. However, RGPs present a limitation shared by most GP approaches: they become computationally infeasible when facing very large datasets. In the present work, with the aim of improving scalability, we modify the original variational approach used with RGPs in order to enable inference via stochastic mini-batch optimization, giving rise to the Stochastic Recurrent Variational Bayes (S-REVARB) framework. We review recent related literature and comprehensively contextualize it with our approach. Moreover, we propose two learning procedures, the Local and Global S-REVARB algorithms, which prevent computational costs from scaling with the number of training samples. The global variant permits even greater scalability by also preventing the number of variational parameters from increasing with the training set, through the use of neural networks as sequential recognition models. The proposed framework is evaluated in the task of dynamical system identification for large scale datasets, a scenario not readily supported by the standard batch inference for RGPs. The promising results indicate that the S-REVARB framework opens up the possibility of applying powerful hierarchical recurrent GP-based models to massive sequential data.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

The analysis of a dynamical system demands that its behavior to be explained by a mathematical model, which can be obtained theoretically or experimentally. The theoretical methodology is based on findings supported by equations relating parameters from a known structure, determined by underlying laws of Physics. On the other hand, the experimental approach, named *identification*, seeks an appropriate model from measurements and *a priori* knowledge, possibly acquired from past experiments (Isermann & Münchhof, 2011). The latter, which avoids the limitations and problem-specific complexities of the purely theoretical approach, is the main application subject of this work.

More specifically, we are interested in applying Gaussian Processes (GP) models to directly describe the probabilistic relations between the observed output data with respect to the inputs, avoiding the fit of a rigid parametrized structure to the data. The Bayesian nature of GP-based models considers the uncertainty as being inherited from the quality of the available data and the modeling assumptions, which enables a clear probabilistic interpretation of its predictions *a posteriori*, i.e., after the training step. Those features have made GPs attractive stochastic tools to applications in control and system identification in general (Kocijan, 2016).

GP-based Nonlinear Autoregressive with eXogenous inputs (GP-NARX) models are among the most common approaches to dynamical modeling with GPs. Since early work in Kocijan, Girard, Banko, and Murray-Smith (2005), Murray-Smith, Johansen, and Shorten (1999) and Solak, Murray-Smith, Leithead, Leith, and Rasmussen (2003), many authors have followed this direction, usually proposing additional features such as nonstationary covariance function for time series prediction (Brahim-Belhouari & Bermak, 2004) and nonstationary system identification (Rottmann & Burgard, 2010), local modeling (Ažman & Kocijan, 2011), integrated pre-processing filter (Frigola-Alcade & Rasmussen, 2013) and system identification in the presence of outliers (Mattos, Santos, & Barreto, 2015). Many authors have also explored GP-based State-Space Models (GP-SSM), using different approaches for approximate inference, such as expectation maximization (EM) for GP dynamical learning (Turner, Deisenroth, & Rasmussen, 2010), use of particle Markov Chain Monte Carlo (PMCMC) (Frigola-Alcade, Lindsten, Schön, & Rasmussen, 2013; Svensson, Solin, Särkkä, & Schön, 2016) and hybridization of variational Bayes and sequential Monte Carlo (SMC) (Frigola-Alcade, Chen, & Rasmussen, 2014).

* Correspondence to: Computer Science Department (DC), Federal University of Ceará (UFC), Center of Sciences, Campus of Pici, Building 910, Fortaleza, Ceará, 60440-900, Brazil.
E-mail addresses: cesarlincoln@dc.ufc.br (C.L.C. Mattos), gbarreto@ufc.br (G.A. Barreto).

https://doi.org/10.1016/j.neunet.2019.01.005
0893-6080/© 2019 Elsevier Ltd. All rights reserved.

Our focus is on the recently introduced Recurrent Gaussian Processes (RGPs) formulation (Mattos, Dai, Damianou, Barreto, & Lawrence, 2017; Mattos et al., 2016), which presents a specifically designed hierarchical dynamical structure with *latent* (i.e. non-observed) variables to represent *internal* dynamics, being related to SSMs. In Mattos et al. (2016), inference with RGP models was tackled by the Recurrent Variational Bayes (REVARB) framework, which handles uncertainty during training and testing steps, the latter via *free simulation*[1] with approximate uncertainty propagation.

An important challenge to the task of modeling lies in the analysis of large quantities of sequential records generated by large scale systems and modern sensing devices, a problem that has recently caught the attention of the dynamical system modeling community (Carli, Chiuso, & Pillonetto, 2012; Cheng, Wang, Camps, & Sznaier, 2015; Green, Cross, & Worden, 2015; Green & Maskell, 2017; Kim, Mallick, Bhowmick, & Chen, 2013; Santos & Barreto, 2017). However, the applicability of GP-based models to such scenarios is limited, since standard GP approaches feature a cubic $\mathcal{O}(N^3)$ complexity with respect to the number of training samples N, besides a $\mathcal{O}(N^2)$ memory requirement.

A commonly used alternative to standard GP models is Titsias' variational sparse framework (Titsias, 2009b), which alleviates such computational and memory demands to $\mathcal{O}(NM^2)$ and $\mathcal{O}(NM)$, respectively, where we can choose $M < N$ *inducing points*.[2] However, the dependency with the number of samples remains and in larger N scenarios even the sparse structure may be infeasible. It should be noted that RGP models with REVARB inference follow strategies from such sparse framework and, hence, present similar poor scalability.

Several works have been presented with the aim of scaling GP models up to handle very large datasets, following diverse approaches, such as the use of mixtures of local experts (Nguyen & Bonilla, 2014), tree-structured approximations (Bui & Turner, 2014), distributed computations (Dai, Damianou, Hensman, & Lawrence, 2014; Deisenroth & Ng, 2015; Gal, van der Wilk, & Rasmussen, 2014; Hoang, Hoang, & Low, 2016), Kronecker and Toeplitz methods with kernel interpolation (Wilson, Dann, & Nickisch, 2015; Wilson & Nickisch, 2015), fast matrix factorization techniques (Ambikasaran, Foreman-Mackey, Greengard, Hogg, & O'Neil, 2016), exploitation of GP spectral representation (Hensman, Durrande, & Solin, 2016) and parametric approximations (Raissi, 2017).

Instead of the aforementioned techniques, which often require parallel implementations, specific matrix structures or standard regression set-ups, we pursue the classic and versatile alternative of using *stochastic* optimization methods to deal with the problem of learning from large scale datasets (Bottou, 1999, 2004; Robbins & Monro, 1951). A seminal work that followed this direction in the context of GP-based models was the one presented by Hensman, Fusi, and Lawrence (2013), who incorporated the general ideas behind stochastic variational inference (SVI) (Hoffman, Blei, Wang, & Paisley, 2013) into the variational sparse GP framework, enabling it to handle hundreds of thousands of samples.

While in the present paper we mainly follow a variational approach (Blei, Kucukelbir, & McAuliffe, 2017; Jordan, Ghahramani, Jaakkola, & Saul, 1999; Wainwright, Jordan, et al., 2008), it should be emphasized that many authors have explored different methods to perform approximate inference with intractable models, such as MCMC and SMC (Kantas, Doucet, Singh, Maciejowski, Chopin, et al., 2015; Neal, 1997; Schön et al., 2015) and expectation propagation (EP) (Gelman et al., 2014; Minka, 2001), where MCMC and SMC follow stochastic sampling techniques and EP, as well as variational methods, is deterministic. However, MCMC algorithms are usually difficult to be applied in very complex models and do not scale well to large datasets (Blei et al., 2017). Scalable EP approaches have been proposed by some authors, such as the one presented in Bui et al. (2016) for regression with Deep GPs (Damianou, 2015). Nevertheless, the empirical results presented in Mattos et al. (2015) in the context of robust system identification, where the variational approach performed better than the EP variant, the recent work on RGPs (Mattos et al., 2017, 2016) and the massively scalable property of the SVI framework (Hensman et al., 2013; Hoffman et al., 2013) encourage us to pursue the use of variational techniques.

Most of the research on scalable GP models have been restricted to standard regression and classification tasks. The dynamical structure of the RGP model introduces latent variables and hierarchical recurrent relations, which demand additional care. Thus, inspired by Hensman et al. (2013) and other recent works, in the present paper we formulate a stochastic version for the original REVARB method, to be called henceforth the S-REVARB framework, which enables the application of the RGP model to large sequential datasets. We derive two algorithms to implement S-REVARB inference, named *Local* and *Global* S-REVARB, which scale up training by enabling stochastic updates with *chunks* (or *mini-batches*) of data. Furthermore, the global variant also incorporates neural networks (NNs) to constrain the model's local variables, avoiding the increase of adjustable parameters when facing growing data and opening up the possibility of applying the powerful hierarchical structure of RGPs to massive datasets.

In summary, the main contributions of the present paper, which will be detailed in the next sections, are the following ones.

- Modification of the original REVARB method in order to obtain a factorized variational lower bound to the marginal log-likelihood of a RGP model, resulting in the S-REVARB framework.
- Proposal of the Local S-REVARB algorithm to apply the SVI strategy to the RGP model, enabling stochastic mini-batch updates, while preserving all the local variational parameters.
- Use of sequential recognition models based on neural networks to constraint the S-REVARB's local variables, giving rise to the Global S-REVARB formulation, which enables even greater scalability.
- Evaluation of the proposed stochastic approaches with large scale system identification tasks, involving up to 95,000 training samples, much more than what could be handled by the original batch REVARB method.

The remaining of the paper is organized as follows. Section 2 summarizes the standard GP modeling framework. Section 3 describes the RGP model structure. Section 4 presents basic concepts of stochastic optimization and reviews recent work on SVI for GP models. Section 5 details the S-REVARB framework, as well as the Local and Global S-REVARB algorithms. Section 7 reports the results of the performed computational experiments. The paper is concluded in Section 8.

2. The GP modeling framework

Given N paired samples $\{\boldsymbol{x}_i, y_i\}_{i=1}^N$, where $\boldsymbol{x}_i \in \mathbb{R}^D$ are D-dimensional inputs and $y_i \in \mathbb{R}$ are observed outputs, we can write in a regression setup

$$y_i = f(\boldsymbol{x}_i) + \epsilon_i^{(y)}, \tag{1}$$

where $f(\cdot)$ is some unknown function and $\epsilon_i^{(y)}$ is the observation noise. The standard GP modeling approach defines a prior distribution over all the possible functions $f(\cdot)$. More specifically, it puts

[1] Procedure where iterative predictions are made based only on exogenous inputs and past predictions.

[2] The reader is referred to Quiñonero-Candela and Rasmussen (2005) and Rasmussen and Williams (2006) to learn more about other sparse GP approaches.

a joint multivariate Gaussian distribution to all the possible points mapped by the function $f(\cdot)$.

Due to the Gaussian marginalization property, we can proceed by analyzing the finite vector of N observations $\boldsymbol{y} \in \mathbb{R}^N$. Let $\boldsymbol{f} \in \mathbb{R}^N$ be the vector of latent function values, which are not directly observed, and considering a Gaussian observation noise $\epsilon_i^{(y)} \sim \mathcal{N}(0, \sigma_y^2)$ with variance σ_y^2, we have (Rasmussen & Williams, 2006)

$$p(\boldsymbol{f}|\boldsymbol{X}) = \mathcal{N}(\boldsymbol{f}|\boldsymbol{0}, \boldsymbol{K}_f), \tag{2}$$

$$p(\boldsymbol{y}|\boldsymbol{f}) = \mathcal{N}(\boldsymbol{y}|\boldsymbol{f}, \sigma_y^2\boldsymbol{I}), \tag{3}$$

$$p(\boldsymbol{y}|\boldsymbol{X}) = \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}, \boldsymbol{K}_f + \sigma_y^2\boldsymbol{I}), \tag{4}$$

where $\boldsymbol{I} \in \mathbb{R}^{N\times N}$ is the identity matrix. In the former expressions we have defined a zero mean vector $\boldsymbol{0} \in \mathbb{R}^N$ for the GP prior $p(\boldsymbol{f}|\boldsymbol{X})$ and used standard Gaussian properties to obtain the marginal likelihood $p(\boldsymbol{y}|\boldsymbol{X})$. The component in the ith row and jth column of the covariance matrix $\boldsymbol{K}_f \in \mathbb{R}^{N\times N}$ associated with the GP prior is calculated by $[K_f]_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$, where $k(\cdot,\cdot)$ is the covariance (or *kernel*) function. Such function is restricted so that it generates a positive semidefinite matrix $\boldsymbol{K}_f$, also called the *kernel* matrix. The exponentiated quadratic kernel, which enforces a certain degree of smoothness, is a common choice and will be applied throughout this paper:

$$k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \sigma_f^2 \exp\left[-\frac{1}{2}\sum_{d=1}^{D} w_d^2(x_{id} - x_{jd})^2\right], \tag{5}$$

where the subindex $d \in \{1, \ldots, D\}$ indicates the dth components of the input vectors.

The kernel hyperparameters are responsible for characterizing the covariance of the model and can be gathered, along with the noise variance σ_y^2, into the vector $\boldsymbol{\theta} = [\sigma_f^2, w_1^2, \ldots, w_D^2, \sigma_y^2]^\top$ to be determined in the model selection step. Such step usually consists in maximizing the marginal log-likelihood $\log p(\boldsymbol{y}|\boldsymbol{X})$ of the observed data and finding the type II maximum likelihood (ML) (Rasmussen & Williams, 2006) solution $\boldsymbol{\theta}_{\text{ML}}$ for the hyperparameters vector:

$$\boldsymbol{\theta}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} \{\log p(\boldsymbol{y}|\boldsymbol{X})\},$$

so that

$$\log p(\boldsymbol{y}|\boldsymbol{X}) = -\frac{N}{2}\log 2\pi - \frac{1}{2}\log|\boldsymbol{K}_f + \sigma_y^2\boldsymbol{I}| - \frac{1}{2}\boldsymbol{y}^\top\left(\boldsymbol{K}_f + \sigma_y^2\boldsymbol{I}\right)^{-1}\boldsymbol{y}, \tag{6}$$

where $|\cdot|$ denotes the determinant of a matrix. We can calculate analytical gradients $\frac{\partial \log p(\boldsymbol{y}|\boldsymbol{X})}{\partial \boldsymbol{\theta}}$ to apply gradient-based optimization methods, such as the BFGS algorithm (Fletcher, 2013).

Prediction for a new input $\boldsymbol{x}_*$, related to the output f_*, is calculated analytically using standard Gaussian distribution conditioning properties:

$$p(f_*|\boldsymbol{y}, \boldsymbol{X}, \boldsymbol{x}_*) = \mathcal{N}(f_*|\mu_*, \sigma_*^2), \tag{7}$$

$$\mu_* = \boldsymbol{k}_{*f}(\boldsymbol{K}_f + \sigma_y^2\boldsymbol{I})^{-1}\boldsymbol{y},$$

$$\sigma_*^2 = K_* - \boldsymbol{k}_{*f}(\boldsymbol{K}_f + \sigma_y^2\boldsymbol{I})^{-1}\boldsymbol{k}_{f*},$$

where $\boldsymbol{k}_{*f} = [k(\boldsymbol{x}_*, \boldsymbol{x}_1), \ldots, k(\boldsymbol{x}_*, \boldsymbol{x}_N)]$, $\boldsymbol{k}_{f*} = \boldsymbol{k}_{*f}^\top$ and $K_* = k(\boldsymbol{x}_*, \boldsymbol{x}_*)$. The predictive distribution of y_* is similar, but the variance is added by the noise variance σ_y^2.

Fig. 1 illustrates a graphical model to represent the relations between the variables previously defined for the standard GP model for noisy regression. The observations are shown as filled nodes and the latent (non-observed) variables as white nodes. Since the inputs $\boldsymbol{x}_i$ are not random, they are shown as deterministic variables. The observation noise is made explicit by the Gaussian noise variables $\epsilon_i^{(y)}$. It is important to highlight some points in Fig. 1: (*i*) an observation y_i is conditionally independent of the other nodes given its associated latent function variable f_i and observation noise $\epsilon_i^{(y)}$; (*ii*) the thick bar that connects the latent variables f_i indicates that all those variables are connected between themselves, which is expected from the definition of the covariance matrix $\boldsymbol{K}_f$; (*iii*) the prediction f_* is part of the same GP prior that models the N training samples.

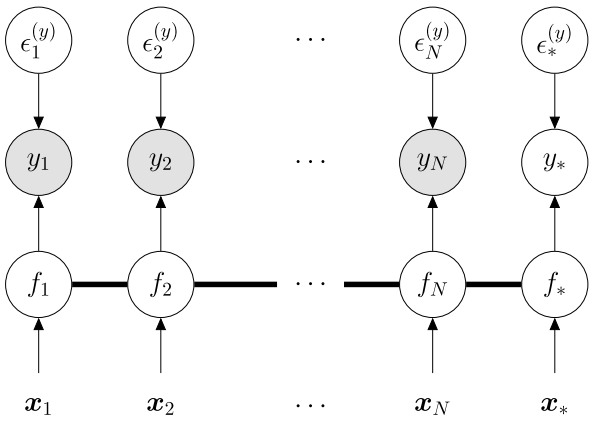


Fig. 1. Graphical models detailing the relation between the variables in a standard GP model. The observations are shown as filled nodes and the latent (non-observed) variables as white nodes. The thick bar that connects the latent variables f_i indicates that all those variables are connected between themselves.

2.1. The GP-NARX model

The GP framework can be directly incorporated to traditional NARX modeling by considering a Gaussian observation noise $\epsilon_i^{(y)}$ and defining regressors for the model input, which gives rise to the GP-NARX model (Kocijan et al., 2005):

$$y_i = f(\boldsymbol{x}_i) + \epsilon_i^{(y)}, \quad \epsilon_i^{(y)} \sim \mathcal{N}(0, \sigma_y^2), \tag{8}$$

$$\boldsymbol{x}_i = [\bar{\boldsymbol{y}}_{i-1}, \bar{\boldsymbol{u}}_{i-1}]^\top = [[y_{i-1}, y_{i-2}, \ldots, y_{i-L_y}], [u_{i-1}, u_{i-2}, \ldots, u_{i-L_u}]]^\top, \tag{9}$$

$$\boldsymbol{f} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{K}_f), \quad [K_f]_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j), \tag{10}$$

where $y_i \in \mathbb{R}$, $\boldsymbol{f} \in \mathbb{R}^N$, $u_i \in \mathbb{R}$ is the exogenous input, σ_y^2 is the noise variance, the model input $\boldsymbol{x}_i \in \mathbb{R}^D$ has dimension $D = L_y + L_u$ and the constants L_y and L_u are respectively the lagged orders chosen for the outputs y_i and inputs u_i. Hyperparameter learning and prediction follow the standard GP regression methodology presented in Section 2.

2.2. The GP-SSM

Models with internal dynamics differentiate themselves from the NARX structure by presenting some form of internal memory. A general formulation can be written in a state-space model (SSM) representation:

$$\boldsymbol{x}_i = f(\boldsymbol{x}_{i-1}, u_{i-1}) + \boldsymbol{\epsilon}_i^{(x)}, \tag{11}$$

$$y_i = g(\boldsymbol{x}_i) + \epsilon_i^{(y)}, \tag{12}$$

where $\boldsymbol{\epsilon}_i^{(x)} \in \mathbb{R}^D$ is the *transition* (or *process*) noise, $f(\cdot)$ is now called the *transition* function, $g(\cdot)$ is the *observation* (or *emission* function) and the vector $\boldsymbol{x}_i \in \mathbb{R}^D$ is called the *state* of the system related to the instant i. The main differences when compared to the previous NARX model are: **(i)** the inclusion of a separate noise

model for the dynamics; **(ii)** the existence of separate functions to model the transition and the observation; **(iii)** the definition of the state $\boldsymbol{x}_i$, which acts as an internal memory with general structure. It is important to emphasize that in most applications the states $\boldsymbol{x}_i$ are *latent*, i.e., not observed directly.

In a GP-SSM context we have the GP priors below (Frigola-Alcade et al., 2013):

$$\boldsymbol{F} \sim \prod_{d=1}^{D} \mathcal{N}(\boldsymbol{f}_{:d}|\boldsymbol{0}, \boldsymbol{K}_f), \quad \text{where } \boldsymbol{f}_{:d} = [\boldsymbol{f}_i]_d|_{i=1}^{N} = F_{id}|_{i=1}^{N}, \tag{13}$$

$$\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{K}_g), \tag{14}$$

where $\boldsymbol{g} \in \mathbb{R}^N$ is the vector of latent outputs from $g(\cdot)$ and $\boldsymbol{F} \in \mathbb{R}^{N\times D}$ is the collection of D latent transition vectors $\boldsymbol{f}_{:d} \in \mathbb{R}^N$, $1 \leq d \leq D$. Note that each output dimension d of $f(\cdot)$, associated with each state dimension, is modeled by a separate GP, although with the same inputs. Moreover, the latent structure of the GP-SSM and the recurrent nature of its states bring several intractabilities to the model, which require the use of approximate inference methods.

3. The recurrent GP framework

Though simple and tractable, the GP-NARX model results in inconsistent noise assumptions and mishandles the uncertainty in its basic formulation. The reason is that , although the input of the GP-NARX contains the random vector $\bar{\boldsymbol{y}}_{i-1}$ (Eq. (9)), the more conventional approaches consider it to be deterministic, similar to the exogenous input $\bar{\boldsymbol{u}}_{i-1}$. Thus, the observation noise in Eq. (8), which is the only modeled noise, should not be independent, i.e., future outputs are actually conditioned on past noisy outputs. Such unrealistic independent noise assumption is a well known limitation of NARX models in general (Nelles, 2013).

On the other hand, GP-SSMs admit non-observed variables, the *states*, and usually separate the modeling of the transition (between states) and observation (from state to the output) functions. However, its definition of the latent states may be vague and ambiguous, since usually there is no strong *a priori* assumptions about them. Besides, the standard and analytical GP approach presented in Section 2 does not apply and approximate methods become necessary, increasing the overall inference complexity when compared to GP-NARX.

Bearing those observations in mind, the RGP (Recurrent GP) framework was proposed in Mattos et al. (2016) as an alternative SSM approach where the states have an autoregressive structure and, unlike standard NARX models, the autoregression is performed with latent variables modeled by probability distributions. Furthermore, RGPs can be extended with the following *deep* recurrent structure, which includes H transition (or *hidden*) layers and one observation (or *emission*) layer, giving rise to the following set of hierarchical expressions:

$$x_i^{(h)} = f^{(h)}\left(\hat{\boldsymbol{x}}_i^{(h)}\right) + \epsilon_i^{(h)}, \qquad 1 \leq h \leq H, \tag{15}$$

$$y_i = f^{(H+1)}\left(\hat{\boldsymbol{x}}_i^{(H+1)}\right) + \epsilon_i^{(H+1)}, \tag{16}$$

$$\boldsymbol{f}^{(h)} \sim \mathcal{N}\left(\boldsymbol{0}, \boldsymbol{K}_f^{(h)}\right), \qquad 1 \leq h \leq H+1,$$

where the unknown functions $f(\cdot)^{(h)}$ have GP priors with zero mean and covariance matrix $\boldsymbol{K}_f^{(h)}$, the noise in each layer is defined as $\epsilon_i^{(h)} \sim \mathcal{N}(0, \sigma_h^2)$ and the upper index differentiates variables and functions from distinct layers. We follow the notation below for the inputs $\hat{\boldsymbol{x}}_i^{(h)}$ of each layer, as introduced in Mattos et al. (2016):

$$\hat{\boldsymbol{x}}_i^{(h)} = \begin{cases} \left[\bar{\boldsymbol{x}}_{i-1}^{(1)}, \bar{\boldsymbol{u}}_{i-1}\right]^\top, & \text{if } h = 1, \\ \left[\bar{\boldsymbol{x}}_{i-1}^{(h)}, \bar{\boldsymbol{x}}_i^{(h-1)}\right]^\top, & \text{if } 1 < h \leq H, \\ \bar{\boldsymbol{x}}_i^{(H)}, & \text{if } h = H+1. \end{cases} \tag{17}$$

where the regressors are defined as $\bar{\boldsymbol{x}}_{i-1}^{(h)} = \left[x_{i-1}^{(h)}, \ldots, x_{i-L}^{(h)}\right]^\top$ and $\bar{\boldsymbol{u}}_{i-1} = [u_{i-1}, \ldots, u_{i-L_u}]^\top$, given the lag orders L and L_u for the model latent dynamical variables $x_i^{(h)}$ and the exogenous inputs u_i, respectively. From Eq. (17) we can see that the domains and ranges of the functions $f(\cdot)^{(h)}$ in Eqs. (15) and (16) are defined as $f^{(1)} : \mathbb{R}^{(L+L_u)} \to \mathbb{R}$, $f^{(H+1)} : \mathbb{R}^L \to \mathbb{R}$ and $f^{(h)} : \mathbb{R}^{(2L)} \to \mathbb{R}$, $1 < h \leq H$.

The vector $\bar{\boldsymbol{x}}_i^{(h)}$ represents the autoregressive latent state related to the layer h in the instant i, from which the dynamics are represented and learned. We emphasize that the RGP model preserves the unobserved states of standard SSMs but avoids the ambiguities of generic multidimensional states by imposing a specific latent autoregressive structure. It is also worth mentioning that the RGP model, as defined by Eqs. (15) and (16), can be seen as a special case of the general Deep GP framework (Damianou, 2015) where the priors of the latent variables in each hidden layer follow the autoregressive structure of Eq. (17). Thus, the RGP model inherits the powerful expressiveness and uncertainty handling properties of Deep GPs. Simplified graphical models[3] for the RGP framework are presented in Fig. 2, where is possible to see in Fig. 2-b the recurrent connections in the transition layers.

Inference is intractable in the RGP model, since the inputs in the GP priors in each layer are now probabilistic latent variables. In order to approximately propagate the uncertainty throughout the model, a variational scheme named REcurrent VARiational Bayes (REVARB) was then proposed in Mattos et al. (2016). REVARB is based on Titsias' variational sparse framework (Titsias, 2009b). It includes to each layer h a number of M inducing points $\boldsymbol{z}^{(h)} \in \mathbb{R}^M$ evaluated in M pseudo-inputs $\boldsymbol{\zeta}_j^{(h)}|_{j=1}^{M} \in \mathbb{R}^{D_h}$, where D_h is the same dimension of $\hat{\boldsymbol{x}}_i^{(h)}$. Such inducing points $\boldsymbol{z}^{(h)}$ are extra samples of the GP that models $f^{(h)}(\cdot)$ and we can write $p\left(\boldsymbol{z}^{(h)}\right) = \mathcal{N}\left(\boldsymbol{z}^{(h)} \middle| \boldsymbol{0}, \boldsymbol{K}_z^{(h)}\right)$, where $\boldsymbol{K}_z^{(h)} \in \mathbb{R}^{M\times M}$ is the covariance matrix computed from $\boldsymbol{\zeta}_j^{(h)}|_{j=1}^{M}$.

In order to obtain a variational lower bound to the marginal log-likelihood of the model, which will then be used to perform model selection, the following factorized variational distribution Q is then introduced:

$$Q = \left(\prod_{h=1}^{H} q\left(\boldsymbol{x}^{(h)}\right)\right)\left(\prod_{h=1}^{H+1} q\left(\boldsymbol{z}^{(h)}\right)\right)\left(\prod_{i=L+1}^{N}\prod_{h=1}^{H+1} p\left(f_i^{(h)} \middle| \boldsymbol{z}^{(h)}, \hat{\boldsymbol{x}}_i^{(h)}\right)\right). \tag{18}$$

The distribution $q\left(\boldsymbol{x}^{(h)}\right)$ related to the latent dynamical variables follows a mean-field approximation:

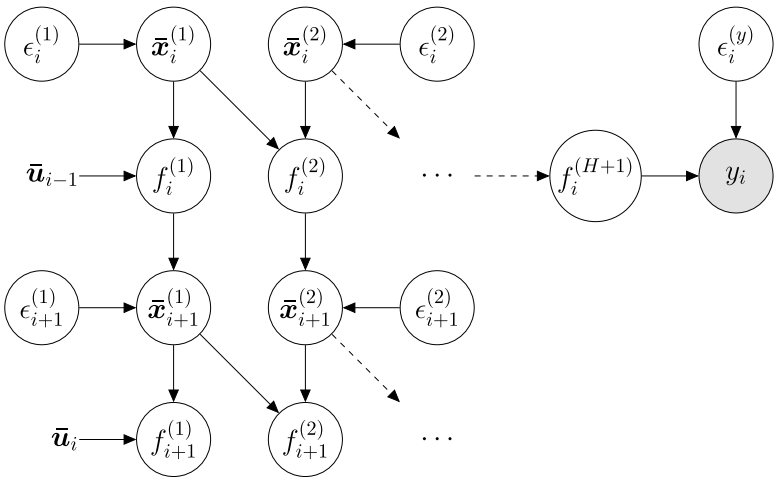
$$q\left(\boldsymbol{x}^{(h)}\right) = \prod_{i=1}^{N} \mathcal{N}\left(x_i^{(h)} \middle| \mu_i^{(h)}, \lambda_i^{(h)}\right), \tag{19}$$

where $\mu_i^{(h)} \in \mathbb{R}$ and $\lambda_i^{(h)} \in \mathbb{R}_{>0}$ are variational parameters that characterize the posterior approximations.

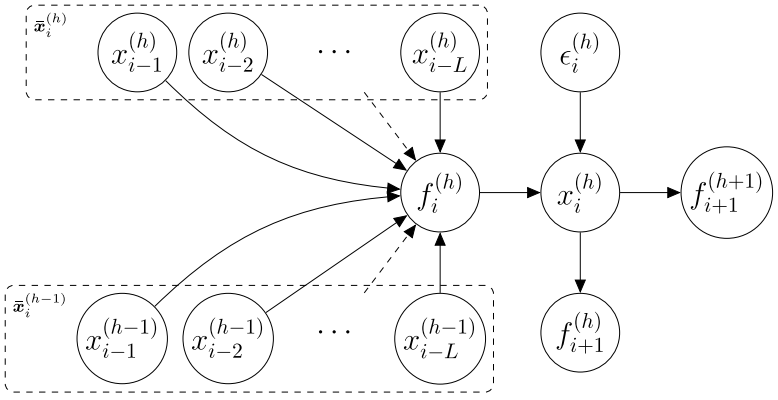
REVARB proceeds by optimally eliminating the distributions $\prod_{h=1}^{H+1} q\left(\boldsymbol{z}^{(h)}\right)$ and obtaining a lower bound that can be maximized guided by analytical gradients, which enables jointly optimization of all the model's hyperparameters and variational parameters.

Promising results were reported in Mattos et al. (2016) for nonlinear system identification and other tasks involving dynamical data. However, the batch REVARB framework does not scale well with the number of training samples, since its computational cost scales proportionally to $\mathcal{O}(NM^2)$ and the memory cost grows with $\mathcal{O}(NM)$, limiting its application to large scale datasets.

[3] We present simplified graphical models showing only the dependencies between the nodes related to the ith observation (ith and $(i+1)$th indexes of the latent variables), which is equivalent to a single column of the full GP diagram presented in Fig. 1.



(a) RGP graphical model with H hidden layers.



(b) Detailing of a single recurrent layer h.

Fig. 2. Simplified graphical models for the RGP framework with H transition layers. Detailing of a single recurrent transition layer h, $1 \leq h \leq H$, is shown in Fig. 2-b. Note that for $h = 1$, the variables $x_{i-l+1}^{(h-1)}|_{l=1}^{L}$ are replaced by the deterministic control inputs $u_{i-l}|_{l=1}^{L_u}$. If these are not available, they are simply omitted. The remaining connections of the nodes $f_{i+1}^{(h)}$ and $f_{i+1}^{(h+1)}$ were omitted in Fig. 2-b, but should be clear from Fig. 2-a.

Aiming at better scalability, in this work we propose the Stochastic REVARB (S-REVARB) framework, which modifies the variational lower bound of the original REVARB in order to enable stochastic updates.

Remark. Similar to some popular artificial neural network models, such as the multilayer perceptron (MLP) and radial basis function (RBF) networks, which were not originally proposed to tackle dynamical data, GP models have been gradually adapted and evaluated in the context of modeling dynamical systems. Seminal works such as Deisenroth, Turner, Huber, Hanebeck, and Rasmussen (2012) have already addressed such viability. The many references mentioned in Section 1 have demonstrated theoretically and empirically the usefulness of such modeling approach via NARX and SSM structures, applying the inherent nonparametric and probabilistic nature of GPs. RGP is one of the latest efforts in such direction, with the focus of approximately propagating uncertainty in both inference and free simulation steps. Furthermore, we refer the readers to the work by Reece and Roberts (2010) and Särkkä, Solin, and Hartikainen (2013) for detailed connections between GP models and Kalman filtering/smoothing methods, classic techniques traditionally applied to dynamical modeling.

4. Stochastic variational inference with GP models

Before describing the S-REVARB framework, it is worth summarizing the stochastic optimization task and the *state of the art* of stochastic variational inference (SVI) for GP models. Arguably, the most common approach to guide an initial candidate towards the solution of an optimization problem involves taking small steps in the direction of noisy gradients as follows:[4]

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} + \alpha_i \boldsymbol{g}_{t-1}, \tag{20}$$

where α_t is a small *learning step* (a.k.a. *learning rate*) and the vector $\boldsymbol{g}_{t-1}$ indicates an unbiased estimation of the gradient of an objective function $L(\cdot)$ that needs to be maximized with respect to the candidate solution $\boldsymbol{\theta}_{t-1}$, i.e., $\boldsymbol{g}_{t-1} \approx \frac{\partial L}{\partial \boldsymbol{\theta}_{t-1}}$. It should be noted that such approximation, although unbiased, may actually be considerably rough.

In the context of stochastic learning, the estimated gradient $\boldsymbol{g}_{t-1}$ related to the parameters update in each optimization step can be computed from a small set (a *mini-batch*) of training samples

[4] Although not the focus of the present work, stochastic gradient-free approaches are also largely present in the literature. We mention for reference metaheuristic algorithms (Talbi, 2009) and Bayesian optimization (Snoek, Larochelle, & Adams, 2012). Interestingly, the latter is usually GP-based.

or even a single data point, which enables learning with very large datasets.

It is widely known that optimization methods stated as in Eq. (20) converge to a local optimum (or a global optimum, if the objective function $L(\cdot)$ is convex), given some mild assumptions, such as that $\sum \alpha_t = \infty$ and $\sum \alpha_t^2 < \infty$ (Bottou, 1999). We refer the readers to Bottou (2004) for a comprehensive review on the importance of stochastic optimization in the machine learning literature.

The original general SVI framework proposed by Hoffman et al. (2013) applies stochastic learning techniques to optimize a variational objective function, such as a lower bound to the model marginal likelihood. SVI follows the simplified steps below:

1. Sample some data points from the training set;
2. Optimize local variational parameters (if they exist);
3. Form intermediate global variational parameters;
4. Stochastically update global variational parameters.

In the above steps, local parameters are those related to *local variables*, i.e., associated to each data sample, while global parameters are those related to *global variables*, i.e., associated to the whole model or some specific model part, such as a layer. It should be noted that the number of local parameters increase with the number of observations, while the number of global parameters do not.

Hoffman et al. applied this approach to learn latent Dirichlet allocation (LDA) and hierarchical Dirichlet process (HDP) topic models from datasets with millions of documents. In the case of the LDA model, the authors enable such scalability by considering the topics as global variables, while the topic proportions (related to each observed document) and topic assignments (related to each observed word in each document) are seen as local variables (Hoffman et al., 2013). In each iteration of the inference step, a few (or even a single) documents are sampled and have their associated local parameters locally optimized. Then, the global parameters (associated with the topics) are stochastically updated using only the sampled local variables. We can see that one key advantage of the SVI approach is that we do not need to analyze the whole dataset before making updates to the variational parameters.

From the aforementioned listed steps, one can note that we have at least two requirements in order to apply SVI: (*i*) a set of global parameters; (*ii*) a factorized variational objective that enables separation of the observations and, if they exist, the corresponding local variational parameters.

Standard GP models do not present any of those requirements. In contrast, Titsias' variational sparse GP framework includes inducing points $\boldsymbol{z}$, which act as global variables. However, the sparse GP equations turn the observations dependent, which can be noticed from the variational sparse lower bound (Titsias, 2009b):

$$\log p(\boldsymbol{y}|\boldsymbol{X}) \geq \log \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}, \sigma_y^2 \boldsymbol{I} + \boldsymbol{K}_{fz}\boldsymbol{K}_z^{-1}\boldsymbol{K}_{fz}^\top) - \frac{1}{2\sigma_y^2}\mathrm{Tr}(\boldsymbol{K}_f - \boldsymbol{K}_{fz}\boldsymbol{K}_z^{-1}\boldsymbol{K}_{fz}^\top), \tag{21}$$

where the elements of the covariance matrices $\boldsymbol{K}_z \in \mathbb{R}^{M\times M}$ and $\boldsymbol{K}_{fz} \in \mathbb{R}^{N\times M}$ are given respectively by $[\boldsymbol{K}_z]_{jj'} = k(\boldsymbol{\zeta}_j, \boldsymbol{\zeta}_{j'})$, $[\boldsymbol{K}_{fz}]_{ij} = k(\boldsymbol{x}_i, \boldsymbol{\zeta}_j)$ and $\boldsymbol{\zeta}_j|_{j=1}^M$ are the pseudo-inputs. The first term on the right side of Eq. (21) is responsible for preventing a factorization along the observations, since it contains the expressions $\log\left|\sigma_y^2\boldsymbol{I} + \boldsymbol{K}_{fz}\boldsymbol{K}_z^{-1}\boldsymbol{K}_{fz}^\top\right|$ and $\left(\sigma_y^2\boldsymbol{I} + \boldsymbol{K}_{fz}\boldsymbol{K}_z^{-1}\boldsymbol{K}_{fz}^\top\right)^{-1}$, which are not factorisable with respect to the training samples. Those remarks were made by Hensman et al. (2013), who noticed that such behavior is result of the exact marginalization of the inducing variables $\boldsymbol{z}$, as performed in Titsias' original framework.

In order to enable SVI in the variational sparse GP framework, Hensman et al. derived a non-collapsed lower bound, i.e., a bound explicitly parametrized by a variational distribution $q(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{m}, \boldsymbol{S})$ as follows (Hensman et al., 2013):

$$\log p(\boldsymbol{y}|\boldsymbol{X}) \geq \sum_{i=1}^{N}\left\{\log \mathcal{N}(y_i|\boldsymbol{k}_i^\top \boldsymbol{K}_z^{-1}\boldsymbol{m}, \sigma_y^2) - \frac{1}{2\sigma_y^2}[\boldsymbol{K}_f]_{ii} - \frac{1}{2}\mathrm{Tr}(\boldsymbol{S}\boldsymbol{\Lambda}_i)\right\} - \mathrm{KL}(q(\boldsymbol{z})||p(\boldsymbol{z})), \tag{22}$$

where $\boldsymbol{k}_i$ is the ith column of $\boldsymbol{K}_{fz}^\top$ and $\boldsymbol{\Lambda} = \frac{1}{\sigma_y^2}\boldsymbol{K}_z^{-1}\boldsymbol{k}_i\boldsymbol{k}_i^\top\boldsymbol{K}_z^{-1}$. Hensman et al. emphasize that the derivatives of this new bound with respect to the moments of $q(\boldsymbol{z})$ show that the optimal values of the moments are obtained when those derivatives are set to zero. Thus, the bound expressed in Eq. (22) is always looser (or equal, if the optimal moments are used) than the original variational sparse lower bound in Eq. (21).

The new variational bound in Eq. (22), factorized along the training samples, enables online (one sample per iteration) or mini-batch updates. The SVI-GP methodology consists in sampling from the training data, computing the noisy derivatives[5] of Eq. (22) with respect to the moments $\boldsymbol{m}$ and $\boldsymbol{S}$, the pseudo-inputs $\boldsymbol{\zeta}_j|_{j=1}^M$ and the kernel hyperparameters, and then proceed by taking a small step in the direction of the computed gradient. We note that in Hensman et al. (2013) the pseudo-inputs are actually kept fixed and the update steps follow the *natural gradient* direction, i.e., the estimated gradient is weighted by the inverse of the Fisher information matrix (Amari, 1998). Later, in Hensman, Matthews, and Ghahramani (2015), such SVI approach was adapted to highly scalable GP models for classification, where it was used to train models with millions of data points. This could no be achieved with either standard or sparse GPs optimized in batch.

Recently, other authors have built upon the original SVI-GP framework to enable further scalability within the SVI context, such as the stochastic variational Kronecker-structured GP named *Blitzkriging* (Nickson, Gunter, Lloyd, Osborne, & Roberts, 2015), automated variational inference for non-Gaussian likelihood (Dezfouli & Bonilla, 2015), a distributed SVI-GP framework (Hoang, Hoang, & Low, 2015) and hybridization with deep neural networks (Wilson, Hu, Salakhutdinov, & Xing, 2016b).

One can see from the SVI-GP bound presented in Eq. (22) that it does not contain local latent variables such as the ones from the GP Latent Variable Model (GP-LVM) formulation (Lawrence, 2004; Titsias & Lawrence, 2010). Hensman et al. comment in Hensman et al. (2013) that a similar factorized bound could be achieved for the GP-LVM and that a SVI approach would be possible by alternating between the optimization of the local variational parameters, i.e., the moments of the variational distribution $q(\boldsymbol{X})$ associated with the latent variables $\boldsymbol{X}$, while keeping the global parameters fixed, and then fixing the values of the moments of $q(\boldsymbol{X})$ and stochastically updating the global parameters. The authors, however, did not present any experimental evaluation of this methodology.

Such experiments were performed by Damianou (2015) in the context of high-dimensional data visualization, where SVI was successfully applied to the Bayesian GP-LVM (Titsias & Lawrence, 2010) in order to be used with datasets containing tens of thousands of samples, following an optimization methodology which includes an adaptive learning rate, firstly proposed in Hensman, Damianou, and Lawrence (2014). Nevertheless, Damianou comments that, although viable, the experiments indicated that *the corresponding optimization procedure is very unstable because certain data batches can cause bifurcation-like effects*.

[5] We emphasize that the noise in the gradients comes from the fact that they are computed only from a subset of the training data in a given iteration of the algorithm.

Bui and Turner (2015) argue that such SVI scheme for models derived from the GP-LVM is not practical for datasets of even modest size, since it can take a long time to converge because each iteration only updates the local variational parameters related to the samples in the current mini-batch and ignores the local optimizations performed in the previous iterations. In order to fix that, they propose the use of recognition models, such as the multilayer perceptron (MLP) network, to model the local variational parameters. In that case the network's parameters (weights) would be shared between all data points and actually behave themselves as global parameters. Bui and Turner illustrate the use of this approach, which does not contain any local parameter, in the task of unsupervised learning, using datasets with up to 18,000 images.

All the aforementioned contributions are not specifically designed to handle dynamical data and do not consider dynamical latent variables, such as the ones introduced by the RGP model. Thus, motivated by those recent works on SVI for GPs and GP-LVMs, we develop a non-collapsed variational lower bound for the REVARB method, which enables stochastic inference with the RGP model in large data scenarios. The henceforward called S-REVARB framework will be presented in the next sections. For a comprehensive review on other approaches to scale GP models we refer the reader to the recent preprint by Liu, Ong, Shen, and Cai (2018).

5. S-REVARB: A stochastic formulation of the REVARB framework

The S-REVARB derivation shares the initial formulation of the original REVARB, initiated in Section 3. It is useful to continue by writing the joint distribution relating all the variables of the RGP model (Mattos et al., 2016):

$$
\begin{aligned}
&p\left(\boldsymbol{y}, \left\{\boldsymbol{x}^{(h)}\right\}\Big|_{h=1}^{H}, \left\{\boldsymbol{f}^{(h)}, \boldsymbol{z}^{(h)}\right\}\Big|_{h=1}^{H+1}\right) = \\
&\left(\prod_{i=L+1}^{N} p\left(y_i\middle|f_i^{(H+1)}\right) p\left(f_i^{(H+1)}\middle|\boldsymbol{z}^{(H+1)}, \hat{\boldsymbol{x}}_i^{(H)}\right)\right. \\
&\left.\prod_{h=1}^{H}\prod_{i=1}^{L} p\left(x_i^{(h)}\middle|f_i^{(h)}\right) p\left(f_i^{(h)}\middle|\boldsymbol{z}^{(h)}, \hat{\boldsymbol{x}}_i^{(h)}\right)\right)\left(\prod_{h=1}^{H+1} p\left(\boldsymbol{z}^{(h)}\right)\right) \\
&\times \left(\prod_{i=1}^{L}\prod_{h=1}^{H} p\left(x_i^{(h)}\right)\right),
\end{aligned}
\tag{23}
$$

where the boldface indexless notation $\boldsymbol{x}^{(h)}$ is used to refer to all variables $x_i^{(h)}$, $\forall i \in \{1, \ldots, N\}$, in a given layer h.

Since the exact marginal likelihood $p(\boldsymbol{y})$ still cannot be computed analytically, it is lower bounded by applying Jensen's inequality, following the standard variational approach (Jordan et al., 1999):

$$
\log p(\boldsymbol{y}) \geq \int_{\boldsymbol{f},\boldsymbol{x},\boldsymbol{z}} Q \log\left[\frac{p\left(\boldsymbol{y}, \left\{\boldsymbol{x}^{(h)}\right\}|_{h=1}^{H}, \left\{\boldsymbol{f}^{(h)}, \boldsymbol{z}^{(h)}\right\}|_{h=1}^{H+1}\right)}{Q}\right],
\tag{24}
$$

where we have used the compact notation[6] $\int_{\boldsymbol{f}}$ as the equivalent of $\int_{-\infty}^{\infty} \mathrm{d}\boldsymbol{f}$ and Q is the factorized variational distribution previously presented in Eq. (18).

[6] Such notation for the marginalization of a random variable is used, for instance, in Barber's book (Barber, 2012). Note that the integral covers all possible values taken by the integrated variables.

The S-REVARB framework modifies the original REVARB approach in order to enable stochastic variational inference with the RGP model. To this end, we first need to factorize the REVARB lower bound across the training observations. Following Hensman et al. (2013), we proceed from Eq. (24) without optimally marginalizing the inducing points $\boldsymbol{z}^{(h)}$ in each layer h. Instead, we maintain the parametrized distributions $q\left(\boldsymbol{z}^{(h)}\right) = \mathcal{N}\left(\boldsymbol{z}^{(h)}\middle|\boldsymbol{m}^{(h)}, \boldsymbol{S}^{(h)}\right), \forall h \in \{1, \ldots, H+1\}$, in order to obtain a non-collapsed bound. Following the conventional approach, the covariance matrices $\boldsymbol{S}^{(h)} \in \mathbb{R}^{M\times M}$ are parametrized by their triangular Cholesky factors.

Replacing the definition of the joint distribution (Eq. (23)) and the factorized variational distribution Q (Eq. (18)) in the Jensen's inequality of Eq. (24), the terms $p\left(f_i^{(h)}\middle|\boldsymbol{z}^{(h)}, \hat{\boldsymbol{x}}_i^{(h)}\right)$ inside the logarithm are canceled out, yielding

$$
\begin{aligned}
\log p(\boldsymbol{y}) \geq &\sum_{i=L+1}^{N}\sum_{h=1}^{H+1} \mathcal{L}_i^{(h)} + \sum_{i=1}^{N}\sum_{h=1}^{H} \mathcal{H}_i^{(h)} + \sum_{i=1}^{L}\sum_{h=1}^{H} \mathcal{L}_{0i}^{(h)} \\
&- \sum_{h=1}^{H+1} \mathrm{KL}\left(q\left(\boldsymbol{z}^{(h)}\right)\middle\|p\left(\boldsymbol{z}^{(h)}\right)\right),
\end{aligned}
\tag{25}
$$

where each term is given by

$$
\begin{aligned}
&\mathcal{L}_i^{(H+1)} = \\
&\left\langle p\left(f_i^{(H+1)}\middle|\boldsymbol{z}^{(H+1)}, \hat{\boldsymbol{x}}_i^{(H+1)}\right)\log p\left(y_i\middle|f_i^{(H+1)}\right)\right\rangle_{q(\boldsymbol{x})q(\boldsymbol{z})}, \\
&\mathcal{L}_i^{(h)} = \left\langle p\left(f_i^{(h)}\middle|\boldsymbol{z}^{(h)}, \hat{\boldsymbol{x}}_i^{(h)}\right)\log p\left(x_i^{(h)}\middle|f_i^{(h)}\right)\right\rangle_{q(\boldsymbol{x})q(\boldsymbol{z})}, \\
&\mathcal{H}_i^{(h)} = -\left\langle \log q\left(x_i^{(h)}\right)\right\rangle_{q(\boldsymbol{x})}, \\
&\mathcal{L}_{0i}^{(h)} = \left\langle \log p\left(x_i^{(h)}\right)\right\rangle_{q(\boldsymbol{x})}, \\
&\mathrm{KL}\left(q\left(\boldsymbol{z}^{(h)}\right)\middle\|p\left(\boldsymbol{z}^{(h)}\right)\right) = \int_{\boldsymbol{z}} q\left(\boldsymbol{z}^{(h)}\right)\log q\left(\boldsymbol{z}^{(h)}\right) \\
&- \int_{\boldsymbol{z}} q\left(\boldsymbol{z}^{(h)}\right)\log p\left(\boldsymbol{z}^{(h)}\right).
\end{aligned}
$$

In the former expressions we have used the notation $\langle\cdot\rangle_{p(\cdot)}$ to indicate expectation with respect to the distributions in the subindex. We omit some of the layer's indexes in such distributions in order to avoid notation clutter, but those agree with the variables inside the expectation. We have also denoted $\mathrm{KL}(\cdot\|\cdot)$ as the Kullback–Leibler (KL) divergence between two distributions.

The distributions $p\left(f_i^{(h)}\middle|\boldsymbol{z}^{(h)}, \hat{\boldsymbol{x}}_i^{(h)}\right)$, inside the terms $\mathcal{L}_i^{(h)}$ of Eq. (25) can be found by using the Gaussian conditioning property, since $\boldsymbol{f}^{(h)}$ and $\boldsymbol{z}^{(h)}$ come from the same GP:

$$
\begin{aligned}
&p\left(f_i^{(h)}\middle|\boldsymbol{z}^{(h)}, \hat{\boldsymbol{x}}_i^{(h)}\right) = \mathcal{N}\left(f_i^{(h)}\middle|\left[\boldsymbol{a}_f^{(h)}\right]_i, \left[\Sigma_f^{(h)}\right]_{ii}\right), \\
&\text{where } \boldsymbol{a}_f^{(h)} = \boldsymbol{K}_{fz}^{(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{z}^{(h)}, \\
&\text{and } \Sigma_f^{(h)} = \boldsymbol{K}_f^{(h)} - \boldsymbol{K}_{fz}^{(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\left(\boldsymbol{K}_{fz}^{(h)}\right)^{\top}.
\end{aligned}
\tag{26}
$$

In the above expression, $[\cdot]_i$ extracts the ith element of a vector, $[\cdot]_{ii}$ indicates the ith diagonal element of a matrix, $\boldsymbol{K}_f^{(h)} \in \mathbb{R}^{(N-L)\times(N-L)}$ is the standard kernel matrix obtained from $\hat{\boldsymbol{x}}^{(h)}$, $\boldsymbol{K}_z^{(h)} \in \mathbb{R}^{M\times M}$ is the sparse kernel matrix calculated from the pseudo-inputs $\boldsymbol{\zeta}^{(h)}$ and $\boldsymbol{K}_{fz}^{(h)} = k(\hat{\boldsymbol{x}}^{(h)}, \boldsymbol{\zeta}^{(h)}) \in \mathbb{R}^{(N-L)\times M}$. Note that we do not need to compute the full covariance matrix $\boldsymbol{K}_f^{(h)}$, but only its $N-L$ diagonal elements, a feature inherited from the sparse approximation.

Since all the expectations necessary to compute Eq. (25) involve Gaussian distributions, it is hence tractable. When solving them, the factorized statistics $\Psi_0^{i(h)} \in \mathbb{R}$, $\Psi_1^{i(h)} \in \mathbb{R}^{M\times 1}$ and $\Psi_2^{i(h)} \in \mathbb{R}^{M\times M}$

appear, which are themselves given by

$$\Psi_0^{i(h)} = \left\langle \left[\boldsymbol{K}_f^{(h)} \right]_{ii} \right\rangle_{q(\boldsymbol{x})}, \tag{27}$$

$$\boldsymbol{\Psi}_1^{i(h)} = \left\langle \left[\boldsymbol{K}_{fz}^{(h)} \right]_{i} \right\rangle_{q(\boldsymbol{x})}, \tag{28}$$

$$\boldsymbol{\Psi}_2^{i(h)} = \left\langle \left[\boldsymbol{K}_{fz}^{(h)} \right]_{i} \left[\boldsymbol{K}_{fz}^{(h)} \right]_{i}^{\top} \right\rangle_{q(\boldsymbol{x})}. \tag{29}$$

These expressions are also tractable for our choice of the exponentiated quadratic kernel (Eq. (5)) and are similar to the ones present in the original REVARB. However, and importantly, they are calculated considering only a single sample or a mini-batch, following the statistics detailed in Appendix B.

We emphasize that the S-REVARB bound in Eq. (25) is fully factorized, since it explicitly separates terms associated with the local variational parameters, related to each observation i, and terms associated only with the model's global parameters. For instance, in the original REVARB bound (Mattos et al., 2016), after solving the expectations related to the hth layer and marginalizing the inducing points $\boldsymbol{z}^{(h)}$, the terms $\log \left| \boldsymbol{K}_z^{(h)} + \frac{1}{\sigma_h^2} \boldsymbol{\Psi}_2^{(h)} \right|$ and $\left(\boldsymbol{K}_z^{(h)} + \frac{1}{\sigma_h^2} \boldsymbol{\Psi}_2^{(h)} \right)^{-1}$ appear. Since the statistic $\boldsymbol{\Psi}_2^{(h)} = \left\langle \left(\boldsymbol{K}_{fz}^{(h)} \right)^{\top} \boldsymbol{K}_{fz}^{(h)} \right\rangle_{q(\boldsymbol{x})}$ must be computed with the whole dataset, the bound cannot be factorized along the data samples. Such dependencies do not occur in S-REVARB's full non-collapsed bound, presented in Appendix A, Eq. (A.1).

Following the general SVI framework (Hoffman et al., 2013), if we consider a mini-batch $\mathcal{B}$, i.e., a set of B sequential indexes sampled from the training data, we can rewrite Eq. (25):

$$\begin{aligned} \log p(\boldsymbol{y}) \geq & \frac{N-L}{B} \sum_{i \in \mathcal{B}} \left[\sum_{h=1}^{H+1} \mathcal{L}_i^{(h)} + \sum_{h=1}^{H} \mathcal{H}_i^{(h)} \right] \\ & + \sum_{i=1}^{L} \sum_{h=1}^{H} \mathcal{L}_{0i}^{(h)} - \sum_{h=1}^{H+1} \mathrm{KL}\left(q\left(\boldsymbol{z}^{(h)}\right) \middle\| p\left(\boldsymbol{z}^{(h)}\right) \right), \end{aligned} \tag{30}$$

whose analytical gradients can be used to perform mini-batch stochastic optimization. Note that the factor $\frac{N-L}{B}$ that scales the terms related to the local variables is necessary to avoid the dominance of the terms associated only with the global parameters, i.e., the KL divergence terms. The intuition presented in Hoffman et al. (2013) is that for $B = 1$ the bound is actually formed by replicating a single observation for the whole training set. In the case of mini-batches, the proportional scaling is followed. Such scaling factor would converge to 1 if all the data was to be used at once, where we would have $B = N - L$. Moreover, the terms $\mathcal{L}_{0i}^{(h)}$ need only to be computed when the mini-batch contains samples which directly depend on the initial conditions, i.e., the indexes $1 \leq i \leq L$.

Table 1 summarizes computational and memory requirements of the RGP/S-REVARB framework when compared to other dynamical GP-based models, where our stochastic approach removes the dependence on the number of training samples N.

5.1. Local S-REVARB: Recurrent SVI

One possible approach to use the S-REVARB non-collapsed bound to estimate the variational parameters of the RGP model follows the iterative strategy suggested in Hensman et al. (2013), similar to the experiments performed in Damianou (2015) in the context of the GP-LVM. This approach is summarized below:

1. Sample a mini-batch of training data points.
2. Keep the values of all the model parameters fixed with the exception of the local variational parameters of $\mathcal{N}\left(x_i^{(h)} \middle| \mu_i^{(h)}, \lambda_i^{(h)}\right)$ related to the sampled points.
3. Optimize the variational parameters of previous step until convergence, using analytical gradients of the lower bound.
4. Fix the values of all the variational means and variances.
5. Perform stochastic update steps for the global parameters, i.e., kernel hyperparameters, pseudo-inputs $\boldsymbol{\zeta}_j^{(h)}|_{j=1}^{M}$ and the moments of $q\left(\boldsymbol{z}^{(h)}\right)$ in all layers.

These steps are repeated until the parameters stop changing significantly or for a fixed amount of iterations. Since Step 3 can be computationally intense, it may actually be executed only in some cycles, or *epochs*, through the dataset, e.g., after a sequence of few epochs, perform local optimization in all iterations of the next epoch. Alternatively, instead of performing Step 3 until convergence, it is possible to consider only a stochastic update as follows:

(3) (alternative) Take a small step in the direction of the lower bound's gradients with respect to the local variational parameters related to the samples in the mini-batch.

In this latter case, which is usually more convenient to implement, the alternative Step 3 should be performed every iteration of the algorithm.

Although the described procedure maintains all the local variables, which still scale with N, they are optimized in mini-batches of size $B \ll N$, greatly reducing the computational cost and memory requirement in each iteration, as presented in Table 1. This enables the RGP model to be trained with much more samples than with standard REVARB. When applying the S-REVARB with the alternative Step 3, it will be named as the *Local S-REVARB* framework, to emphasize the explicit presence of the local latent variables.

Remark. It should be noted that the Local S-REVARB framework does not correspond to a new model. It considers the same RGP model introduced in Mattos et al. (2016), including all its probabilistic distributions and variables. Thus, the Local S-REVARB is proposed as a stochastic and more scalable alternative inference method to the original REVARB.

5.2. Global S-REVARB: Sequential recognition models for S-REVARB

Since the local variational parameters related to each training sample are still present in the solution detailed so far, it may suffer from the issues argued in Bui and Turner (2015), where those variables would be stochastically updated at most once per epoch,

Table 1
Comparison of computational and memory requirements of some GP-based dynamical models with respect to the number of training samples N, the number of pseudo-inputs M, the mini-batch size B, the number of hidden layers H and the latent states dimension D (for the GP-SSM from Eleftheriadis et al., 2017).

	Computational	Memory
GP-NARX	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$
Var. Sparse GP-NARX	$\mathcal{O}(NM^2)$	$\mathcal{O}(NM)$
GP-SSM from (Eleftheriadis et al., 2017)[a]	$\mathcal{O}(NDB)$	$\mathcal{O}(NMDB)$
GP-LSTM from (Al-Shedivat, Wilson, Saatchi, Hu, & Xing, 2017)[a]	$\mathcal{O}(N)$	$\mathcal{O}(NM)$
RGP/REVARB	$\mathcal{O}((H+1)NM^2)$	$\mathcal{O}((H+1)NM)$
RGP/S-REVARB	$\mathcal{O}((H+1)BM^2)$	$\mathcal{O}((H+1)BM)$

[a]The computational and memory requirements were either informed in the referred paper or estimated from the equations presented in the related work.

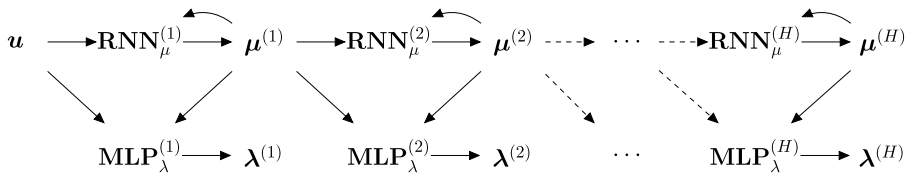


Fig. 3. Diagram for the MLP and RNN-based recognition models of the Global S-REVARB framework in a RGP model with H recurrent layers. Note that the outputs of the RNNs are used as inputs in the MLPs in the same and in the next hidden layer, which must be accounted when backpropagating gradients during training.

i.e., each local optimization does not incorporate any knowledge of other local updates. That could turn the optimization procedure slow and overall inefficient. Bui and Turner fix this issue by using MLP neural networks as recognition models. However, those do not directly consider dynamical data or the RGP structure.

In the case of S-REVARB, we can use the RNN sequential recognition model introduced in Mattos et al. (2016) to generate the local variational parameters. In Mattos et al. (2016) it was applied in a batch context and only for the variational means. Here we are interested in the stochastic scenario and in also applying a separate recognition model for the local variances. Thus, for a given hidden layer $1 \leq h \leq H$, both recognition models are written as follows:

$$q\left(x_i^{(h)}\right) = \mathcal{N}\left(x_i^{(h)} \middle| \mu_i^{(h)}, \lambda_i^{(h)}\right), \tag{31}$$

$$\mu_i^{(h)} = g_\mu^{(h)}\left(\hat{\boldsymbol{x}}_{i-1}^{(h)}\right) = \phi_{\mu,2}\left(\boldsymbol{W}_{\mu,2}^{(h)\top}\phi_{\mu,1}\left(\boldsymbol{W}_{\mu,1}^{(h)}\hat{\boldsymbol{x}}_{i-1}^{(h)}\right)\right), \tag{32}$$

$$\lambda_i^{(h)} = g_\lambda^{(h)}\left(\hat{\boldsymbol{x}}_{i-1}^{(h)}\right) = \phi_{\lambda,2}\left(\boldsymbol{W}_{\lambda,2}^{(h)\top}\phi_{\lambda,1}\left(\boldsymbol{W}_{\lambda,1}^{(h)}\hat{\boldsymbol{x}}_{i-1}^{(h)}\right)\right), \tag{33}$$

where matrices $\boldsymbol{W}_{\mu,l}^{(h)}$ and $\boldsymbol{W}_{\lambda,l}^{(h)}$, $l \in \{1, 2\}$, are the networks' weights, and $\boldsymbol{\phi}_{\mu,l}(\cdot)$ and $\boldsymbol{\phi}_{\lambda,l}(\cdot)$ denote element-wise activation functions. In the current work we use networks with depth equal 2, i.e., with 1 hidden layer, but deeper networks could also be applied. The only restriction is that $g_\lambda^{(h)}$ must generate only non-negative values, since they are related to the variational variances. It is worth mentioning that the latent variables in the input $\hat{\boldsymbol{x}}_{i-1}^{(h)}$ of the recognition models above are in practice replaced by their associated variational means $\mu_i^{(h)}$, since standard NNs do not directly handle probabilistic variables as inputs.

Fig. 3 illustrates the structure and connections of the recognition models that generate the variational means and variances in the hidden layers of the RGP model in this new approach. It is worth noticing that the recognition model related to the variational means expressed in Eq. (32) is a recurrent neural network (RNN), while the model related to the variational variances in Eq. (33) is a regular feedforward MLP network. The MLP is trained with standard backpropagation (Rumelhart, Hinton, & Williams, 1985), while the update of the RNN's parameters is made via the truncated backpropagation through time (TBPTT) algorithm (Williams & Zipser, 1995). Note that the outputs of the RNN are used as inputs in the MLP, so the gradients should be correctly propagated from one network to the other. Furthermore, when more than one hidden layer is present in the RGP model (e.g. 2 hidden recurrent layers), the gradients of a given layer should be backpropagated to the previous layer, following the dependencies shown in Fig. 3.

Importantly, the weights of the NNs are shared among the variational parameters, working as global parameters themselves. This is made clear when we recall that a stochastic update on the weights has effect in the output of the networks given any input. Such sequential recognition models when applied along S-REVARB's non-collapsed bound results in the *Global* S-REVARB framework, which enables mini-batch updates via stochastic gradient ascent. Since now the model contains only global variables, which do not scale with the number of samples, this approach opens up the possibility of performing inference with the RGP model in scenarios with even larger datasets.

Remark. At a first glance, one could misinterpret the Global S-REVARB approach as a parametric version of the RGP model, due to the inclusion of the NN-based recognition models. However, this is not true. First, the obtained recurrent model is very different from a stand-alone parametric RNN, since it is built from the original RGP probabilistic modeling formulation by using the same analytical lower bound of the Local S-REVARB. Second, the sequential recognition models only act as *constraints* to the already analytical nonparametric bound, functioning as variational (and not model) parameters. Similar arguments were also made in Dai, Damianou, González, and Lawrence (2016) in the context of auto-encoded deep GP models.

5.3. Making predictions with the S-REVARB framework

In order to make predictions in the S-REVARB framework we can pursue the methodology presented in Girard, Rasmussen, Quiñonero-Candela, and Murray-Smith (2003) in the context of multi-step ahead GP predictions, which was also used in the original REVARB formulation.

Given a new input $\hat{\boldsymbol{x}}_*^{(h)}$ in the hth layer, we want to approximate the predictive distribution of the associated output $f_*^{(h)}$. First, we recall the following conditional distribution of the variational sparse GP framework, written below for the hth layer:

$$\begin{aligned} p\left(f_*^{(h)} \middle| \hat{\boldsymbol{x}}_*^{(h)}\right) &= \mathcal{N}\left(f_*^{(h)} \middle| \varrho_*^{(h)}, \varsigma_*^{(h)}\right), \\ \varrho_*^{(h)} &= \boldsymbol{k}_{*z}^{(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{m}^{(h)}, \\ \varsigma_*^{(h)} &= K_*^{(h)} - \boldsymbol{k}_{*z}^{(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{k}_{z*}^{(h)} \\ &\quad + \boldsymbol{k}_{*z}^{(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{S}^{(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{k}_{z*}^{(h)}. \end{aligned} \tag{34}$$

Note that the former conditional distribution contains the moments of the variational posterior $q\left(\boldsymbol{z}^{(h)}\right) = \mathcal{N}\left(\boldsymbol{z}^{(h)} \middle| \boldsymbol{m}^{(h)}, \boldsymbol{S}^{(h)}\right)$, which in the case of S-REVARB are left explicitly parametrized by the moments $\boldsymbol{m}^{(h)}$ and $\boldsymbol{S}^{(h)}$, treated as variational parameters. We then proceed to take the expected value of the variable $\hat{\boldsymbol{x}}_*^{(h)}$ by computing the Gaussian approximation of the final predictive distribution in each layer:

$$p\left(f_*^{(h)}\right) = \left\langle p\left(f_*^{(h)} \middle| \hat{\boldsymbol{x}}_*^{(h)}\right)\right\rangle_{q(\boldsymbol{x}_*)} \approx \mathcal{N}\left(f_*^{(h)} \middle| \mu_*^{(h)}, \lambda_*^{(h)}\right), \tag{35}$$

with $\mu_*^{(h)}$ and $\lambda_*^{(h)}$ defined as

$$\mu_*^{(h)} = \left\langle \varrho_*^{(h)}\right\rangle_{q(\boldsymbol{x}_*)} = \left(\boldsymbol{B}^{(h)}\right)^\top\left(\boldsymbol{\Psi}_{1*}^{(h)}\right)^\top, \tag{36}$$

$$\begin{aligned} \lambda_*^{(h)} &= \left\langle \varsigma_*^{(h)}\right\rangle_{q(\boldsymbol{x}_*)} + \mathbb{V}_{q(\boldsymbol{x}_*)}\left\{\varrho_*^{(h)}\right\}, \\ &= \left(\boldsymbol{B}^{(h)}\right)^\top\left(\boldsymbol{\Psi}_{2*}^{(h)} - \left(\boldsymbol{\Psi}_{1*}^{(h)}\right)^\top\boldsymbol{\Psi}_{1*}^{(h)}\right)\boldsymbol{B}^{(h)} + \Psi_{0*}^{(h)} \\ &\quad - \mathrm{Tr}\left(\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\left(\boldsymbol{I} - \boldsymbol{S}^{(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\right)\boldsymbol{\Psi}_{2*}^{(h)}\right), \end{aligned} \tag{37}$$

where we have used properties of conditional distributions, as in Girard, Rasmussen, Quiñonero-Candela, and Murray-Smith (2002), $q(\boldsymbol{x}_*)$ (with omitted layer index) is the approximate distribution of the latent variables in the input $\hat{\boldsymbol{x}}_*^{(h)}$ (similar to the Eq. (17))

and $\mathbb{V}_{q(\boldsymbol{x}_*)}\{\cdot\}$ indicates the variance operator with respect to the distribution $q(\boldsymbol{x}_*)$. We have also defined the matrix

$$\boldsymbol{B}^{(h)} = \left(\boldsymbol{K}_z^{(h)}\right)^{-1} \boldsymbol{m}^{(h)}. \tag{38}$$

The new statistics $\Psi_{0*}^{(h)}$, $\boldsymbol{\Psi}_{1*}^{(h)}$ and $\boldsymbol{\Psi}_{2*}^{(h)}$ are computed using $q\left(\boldsymbol{x}_*^{(h)}\right) = \mathcal{N}\left(\boldsymbol{x}_*^{(h)} \middle| \mu_*^{(h)}, \lambda_*^{(h)} + \sigma_h^2\right)$, $K_*^{(h)} = k\left(\hat{\boldsymbol{x}}_*^{(h)}, \hat{\boldsymbol{x}}_*^{(h)}\right)$ and the vector $\boldsymbol{k}_{*z}^{(h)} = \left[k\left(\hat{\boldsymbol{x}}_*^{(h)}, \boldsymbol{\zeta}_1^{(h)}\right) \cdots k\left(\hat{\boldsymbol{x}}_*^{(h)}, \boldsymbol{\zeta}_M^{(h)}\right)\right]$, similar to the original statistics defined in the variational lower bound. Finally, for the output layer we have $\mathbb{E}\{y_*\} = \mu_*^{(H+1)}$ and $\mathbb{V}\{y_*\} = \lambda_*^{(H+1)} + \sigma_{H+1}^2$.

It is worth emphasizing that the main difference between REVARB and S-REVARB in terms of the predictive step is that the latter, which considers a non-collapsed lower bound, applies the variational parameters $\boldsymbol{m}^{(h)}$ and $\boldsymbol{S}^{(h)}$ that explicitly parametrize the distribution $q\left(\boldsymbol{z}^{(h)}\right)$ to compute the predictive approximations. On the other hand, the original REVARB is able to apply the optimal moments of $q\left(\boldsymbol{z}^{(h)}\right)$, which are derived in its collapsed bound.

The aforementioned predictive equations can be readily applied by the Local S-REVARB, since it uses the local variational distribution $q\left(\boldsymbol{x}_*^{(h)}\right)$ to compute the recursive predictions. In the case of the Global S-REVARB variant, we can describe at least two strategies for making predictions, listed as follows:

- **Variational sparse simulation:** The Local S-REVARB is able to approximately propagate the uncertainty during predictions by sequentially computing Eqs. (36) and (37) for each layer. The Global S-REVARB can also pursue the same methodology, named henceforth *variational sparse simulation*. Such strategy directly uses the moments of the distribution $q\left(\boldsymbol{z}^{(h)}\right) = \mathcal{N}\left(\boldsymbol{z}^{(h)} \middle| \boldsymbol{m}^{(h)}, \boldsymbol{S}^{(h)}\right)$, from the variational sparse approximation, and the predictive variational distribution $q\left(\boldsymbol{x}_*^{(h)}\right)$. Note that in this approach the recognition networks are not used for making predictions.
- **Recognition-based simulation:** Since the Global S-REVARB includes NNs and RNNs, we have an alternative method to perform predictions. It works as follows: the predictive means and variances in the hidden layers are directly computed using the recognition models using Eqs. (32) and (33). Then, in the output layer, the final prediction is made based on Eqs. (36) and (37), following the aforementioned variational sparse simulation approach. Such procedure takes advantage of the already learned sequential recognition models. However, it only considers the predicted variances of the last hidden layer ($h = H$) to perform the final prediction in the output layer. Thus, when using such *recognition-based simulation*, the uncertainty is computed locally but not propagated through the recurrent layers, only in the final observation layer.

After some preliminary experiments, we could not decide clearly which methodology is the best one for performing predictions with the Global S-REVARB framework. Thus, we will explicit which approach was used for each experiment with the Global S-REVARB in the next sections.

Algorithm 1 summarizes the general application of the S-REVARB framework to the RGP model. Both Local and Global versions, i.e., without or with sequential recognition models, follow the same general steps.

Algorithm 1 S-REVARB for stochastic dynamical modeling with the RGP model.

- Estimation step

Require: $\boldsymbol{u} \in \mathbb{R}^N$ (control input vector), $\boldsymbol{y} \in \mathbb{R}^N$ (output vector), H (number of hidden layers), M (number of inducing points), L (latent order lag), L_u (input order lag), B (mini-batch size)
Initialize kernel hyperparameters and variational parameters;
repeat
  Sample B sequential points of the data;
  Compute the marginal log-likelihood lower bound with Eq. (A.1) considering only the sampled points (scaling as in Eq. (30));
  Compute the analytical gradients of Eq. (A.1) with respect to the unknown parameters;
  Update parameters with a stochastic gradient method;
until convergence (or maximum number of iterations) is reached.
Output the optimized model;

- Free simulation with test data

Require: Test external inputs $\boldsymbol{u}_* \in \mathbb{R}^{N_*}$ and the previously estimated RGP model
for $i = 1 : N_*$ **do**
  for $h = 1 : H$ **do**
    Compute the predictive mean $\mu_{*i}^{(h)}$ and variance $\lambda_{*i}^{(h)}$ following one of the strategies presented in Section 5.3;
    Update the variational distributions of the new latent dynamical variable with $q\left(x_{*i}^{(h)}\right) = \mathcal{N}\left(x_{*i}^{(h)} \middle| \mu_{*i}^{(h)}, \lambda_{*i}^{(h)} + \sigma_h^2\right)$;
  end for
  Compute the predictive mean $\mu_{*i}^{(H+1)}$ and variance $\lambda_{*i}^{(H+1)}$ of the output layer using Eqs. (36) and (37);
  Output $y_{*i} \sim \mathcal{N}\left(\mu_{*i}^{(H+1)}, \lambda_{*i}^{(H+1)} + \sigma_{H+1}^2\right)$;
end for

6. Relations with recent GP methods for large and/or dynamical data

After introducing the S-REVARB approach, our contribution towards scaling RGPs, we shall now discuss its relation with other recent GP-based methods proposed in the context of sequential data and/or large training sets.

Wilson et al. (2016b) have introduced a SVI formulation for the deep kernel learning framework (Wilson, Hu, Salakhutdinov, & Xing, 2016a), where a deep neural network is used to parametrize a kernel function whose parameters are learned jointly through the marginal likelihood of the GP. Scalability is obtained via stochastic gradient training and efficient computations by leveraging kernel interpolation techniques and Kronecker product of Toeplitz matrices (Wilson et al., 2015; Wilson & Nickisch, 2015). Experiments are performed in a wide range of classification problems. Although that work does not consider the task of learning from sequences, it should be emphasized that the proposed S-REVARB could also incorporate kernel interpolation and matrix structure exploring techniques in its own computational steps. We leave such promising direction for future work.

Al-Shedivat et al. (2017) have extended the deep kernel approach in Wilson et al. (2016a) to the problem of learning from dynamical data using a LSTM (Long Short-Term Memory) network (Hochreiter & Schmidhuber, 1997) to parametrize the kernel function, which yields the GP-LSTM model. The main difference between the RGP and the GP-LSTM models is that the former performs inference over random recurrent transition functions in its dynamical layers, while the latter considers a deterministic transformation for the input sequence, following a one-step-ahead structure. Although the GP-LSTM does not propagate the uncertainty in the recurrent layer, it is able to represent some randomness in its predictions by including a dropout step (Gal & Ghahramani, 2016; Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov, 2014), which acts as both a regularizer and an uncertainty estimator with low computational effort, at the cost of an additional hyperparameter, the dropout rate. Furthermore, in the GP-LSTM, and also in Wilson et al. (2016b), the neural network is part of the model, directly transforming the input, while in the Global S-REVARB the neural networks are used as recognition models in the inference step.

Our proposed S-REVARB methodology is also distinct from the GP-LSTM training algorithm. In Al-Shedivat et al. (2017), the authors apply a semi-stochastic alternating gradient descent method

with asynchronous optimization steps, where the kernel matrix is precomputed on the full data and fixed, being recomputed only after a pass through the entire data. The S-REVARB strategies we have detailed in Sections 5.1 and 5.2 consider fully stochastic updates, without the need of computing with the full data. That was made possible by using an analytical fully factorized variational lower bound on the true marginal likelihood of the model, an approximation that the GP-LSTM does not require. The RGP/S-REVARB framework achieves scalability via fully stochastic updates using mini-batches, which removes the number of training observations N from the complexity costs (see Table 1). In Al-Shedivat et al. (2017), besides the semi-stochastic strategy, the authors use the same kernel interpolation techniques of Wilson et al. (2016b) to enable fast computations. Such feature turns GP-LSTM faster than S-REVARB in general. However, as we have previously argued, the same matrix structure exploring and interpolation techniques could also be added to S-REVARB computations in the future.

The experimental findings presented in Al-Shedivat et al. (2017) for free simulation scenarios showed that the RGP obtained the best results when compared with all the other evaluated models, including the GP-LSTM. The authors comment that the result was expected, since *RGP was particularly designed to represent and propagate uncertainty through a recurrent process* and that the GP-LSTM framework aims to *build expressive kernels for regression on sequences* (Al-Shedivat et al., 2017). Since in the current work we still focus on free simulation, we do not perform further experiments on GP-LSTM.

Another interesting recent work on dynamical GP models was presented by in Eleftheriadis et al. (2017). The authors consider a general GP-SSM and propose variational approximations to perform inference. Their framework assumes that the posterior of the state transition distribution can be locally linear approximated in a Gauss–Markov structure. Moreover, bi-directional recurrent neural networks are used as recognition models to estimate the variational parameters of the approximated posterior. Efficient computation is obtained via stochastic updates that use the reparameterization trick (Kingma & Welling, 2014; Rezende, Mohamed, & Wierstra, 2014).

The resulted methodology in Eleftheriadis et al. (2017) has some similarities with our S-REVARB approach, such as the use of recognition models in the variational inference step. However, the sampling technique performed by the authors generates in each draw a full data sequence (or *trajectory*) through recursive affine transformations, which scales with the sequence length. Experiments in Eleftheriadis et al. (2017) were performed using only datasets with up to a few thousands observations (considering multiple sequences) and individual sequences with length up to 512. Furthermore, it should be noted that the local linear consideration for the posterior is only valid for transition steps sufficiently close, a restriction not imposed by the S-REVARB. One advantage of the framework proposed in Eleftheriadis et al. (2017) is that it is not restricted to the exponentiated quadratic kernel (Eq. (5)). The latter feature is another interesting direction for improving S-REVARB in future investigations.

7. Computational experiments

In this section we will evaluate the RGP/S-REVARB solution in the task of system identification with large scale datasets. More specifically, we use the *Silverbox* dataset and the *Wiener–Hammerstein* benchmarking datasets. We also perform an initial example with a smaller dataset (the *Damper* dataset) in order to analyze better the proposed algorithms. Table 2 presents basic statistics of the aforementioned datasets, while Fig. 4 shows lagged input–output cross-correlation and output autocorrelation. All our computational implementations were made using the R language (R Core Team, 2017).

Table 2
Statistics of the datasets used in the computational experiments. Note that the reported values are related to the original data, before normalization. Further descriptions and references for the data are presented in the text.

Training data	*Damper*	*Silverbox*	*Wiener–Hammerstein*
Samples	2000	91,072	95,000
Output mean	2.3990	0.0008	−0.0407
Output standard deviation	58.6728	0.0538	0.2451
Input mean	0.0862	0.0062	−0.0039
Input standard deviation	6.5441	0.0219	0.6763
Testing data	***Damper***	***Silverbox***	***Wiener–Hammerstein***
Samples	1499	40,000	84,000
Output mean	−3.1121	0.0008	−0.0403
Output standard deviation	55.5339	0.0529	0.2446
Input mean	−0.1175	0.0062	−0.0036
Input standard deviation	5.4421	0.0220	0.6761

Evaluation is done by calculating the root mean squared error (RMSE) of the free simulation on the test data, given by $\text{RMSE} = \sqrt{\frac{1}{N_*}\sum_{i=1}^{N_*}(y_i - \tilde{\mu}_i)^2}$, where N_* is the number of test samples, y_i is the true test output and $\tilde{\mu}_i$ is the predicted mean output. We also compute the average negative log-predictive density (NLPD), where lower values are preferable, calculated by $\text{NLPD} = \frac{1}{2}\log 2\pi + \frac{1}{2N_*}\sum_{i=1}^{N_*}\left[\log \tilde{\sigma}_i^2 + \frac{(y_i-\tilde{\mu}_i)^2}{\tilde{\sigma}_i^2}\right]$, where $\tilde{\sigma}_i^2$ is the ith predicted variance. We perform single runs in the experiments, similar to other works that apply scalable GP methods to large datasets (Bui et al., 2016; Salimbeni & Deisenroth, 2017). We emphasize however that, given the initialization scheme recommended below, additional runs do not change much.

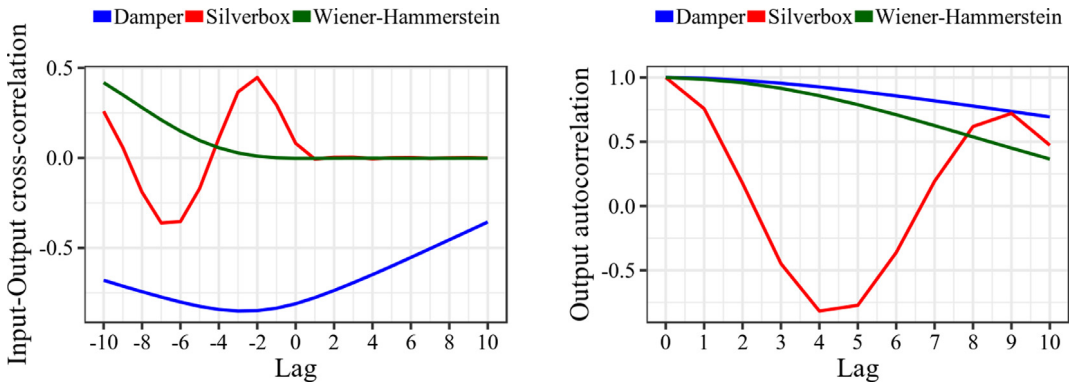
7.1. Implementation details

As follows we present some implementation details we have adopted in the computational experiments.

Mini-batch sampling Since we deal with sequential training data, the mini-batch sampling procedure differs from the one followed by standard non-sequential methods. In our case, we slice the input and output time series with the size of the mini-batch (B) and shuffle the slices before presenting to the optimization algorithm. The important difference lies in the fact that we cannot shuffle the samples within the same mini-batch, in order to preserve the information about the system dynamics.

Activation functions We opted to use different activation functions for each layer in the recognition models of the Global S-REVARB. More specifically, the MLP that models the latent variances applies the hyperbolic tangent in the hidden units, which is given by $f(x) = \frac{\exp(2x)-1}{\exp(2x)+1}$, while the so-called SoftPlus activation function is used in the output units, which is given by $f(x) = \log(1 + \exp(x))$. The latter function ensures that the recognition model will output only positive values for the variances.

The RNN that models the variational means applies the ReLu (Rectified Linear Unit) activation function in the hidden units, which is simply given by $f(x) = \max(0, x)$ and is known for preventing vanishing or exploding gradient problems, which may rise in recurrent networks (Glorot, Bordes, & Bengio, 2011). The output units apply the simple linear function $f(x) = x$, the usual choice in non-restricted regression tasks.



(a) Input-output cross-correlation. (b) Output autocorrelation.

Fig. 4. Input–output cross-correlation and output autocorrelation for the 3 datasets used in the computational experiments.

Model initialization In order to initialize the Global S-REVARB's recognition models, we follow the common approach of sampling the initial weights from the scaled Gaussian distribution $\sqrt{\frac{1}{N_{\text{in}}}}\mathcal{N}(0, 1)$, where N_{in} is the number of inputs of the correspondent neuron. Since we use ReLu units in the hidden layer of the RNN models, we initialize their weights slightly different, sampling from $\sqrt{\frac{2}{N_{\text{in}}}}\mathcal{N}(0, 1)$, which follows the recommendation made by He, Zhang, Ren, and Sun (2015). Moreover, although we initialize all the neurons' biases with zeros, the biases of the ReLu units are initialized with small values, e.g., 0.01, in order to prevent "dead" units, i.e., which always output zero values, in the beginning of the optimization.

The components of the initial pseudo-inputs $\boldsymbol{\zeta}_j^{(h)}|_{j=1}^{M}$ are sampled from the normal distribution $\mathcal{N}(0, 1)$, while the moments of the variational distributions $q\left(\boldsymbol{z}^{(h)}\right)$ are initialized with zero mean vectors and unitary diagonal covariance matrices, i.e., $\boldsymbol{m}^{(h)} = \boldsymbol{0}$ and $\boldsymbol{S}^{(h)} = \boldsymbol{I}$ at the start of the optimization. The kernel hyperparameters are initialized as follows: $\left(\sigma_f^{(h)}\right)^2 = 1$, $\left(w_d^{(h)}\right)^2 = 0.01$, $\sigma_h^2 = 0.01$. A jitter initialized with 0.01 is added to the diagonal of the sparse kernel matrices $\boldsymbol{K}_z^{(h)}$ in each layer and optimized along with the other kernel hyperparameters, as suggested in Titsias (2009a).

The initialization of the variational means $\boldsymbol{\mu}^{(h)}$ is made with information from outputs and inputs. Thus, we initialize the means in all the hidden layers as $\boldsymbol{\mu}^{(h)} = [\boldsymbol{y}, \boldsymbol{u}]\boldsymbol{p}_1$, where the vector $\boldsymbol{p}_1 \in \mathbb{R}^2$ is the principal component of the matrix $[\boldsymbol{y}, \boldsymbol{u}] \in \mathbb{R}^{N\times 2}$. Such initialization corresponds to the Principal Component Analysis (PCA) projection of the data in the dimension of $\boldsymbol{\mu}^{(h)}$. The variational variances can be initialized with values in the interval [0.01, 0.5]. We usually choose $\lambda_i^{(h)} = 0.1, \forall i$, in the recurrent layers.

Optimization strategy There is a vast literature with algorithmic techniques and recommendations related to stochastic gradient-based optimization methods. We use the ADAM optimizer, an adaptive strategy to the learning rate with separate values to each parameter being optimized (Kingma & Ba, 2015).

Moreover, we decay the learning rate α_t in each t iteration of the optimization. More specifically, after 30% of the total iterations we start to exponentially decay α_t, from the initial value of α_{init} until the final value of α_{final}. We found empirically that the values $\alpha_{\text{init}} = 0.02$ and $\alpha_{\text{final}} = 0.002$ work well in practice, but those might need further tuning for other datasets.

The TBPTT algorithm used to train the RNN within the Global S-REVARB follows gradients backpropagated across all samples of a given mini-batch and the previous 30 samples, counted backwards from the first sample in the mini-batch.

Finally, we keep fixed the initial values of the kernel hyperparameters for the initial iterations, e.g., the first 30% of the optimization steps, following similar recommendation in Hensman et al. (2013).

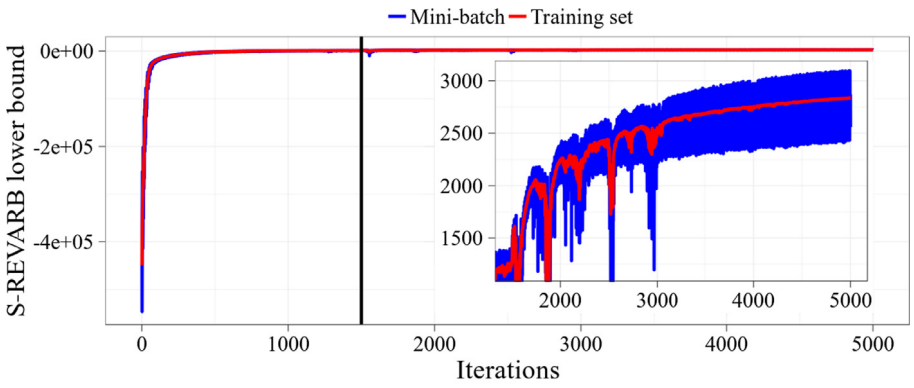
Table 3
RMSE and NLPD values for the free simulation results on the *Damper* dataset. The first five experiments were reported in The MathWorks Inc. (2016). The reduced-rank GP-SSM experiment was presented in Svensson et al. (2016). The experiment with SISOG was reported in Bijl et al. (2016). The NA entry indicates that the NLPD value was not reported. The Global S-REVARB followed recognition-based simulation.

	RMSE	NLPD
Linear OE model (4th order)	27.1	–
Hammerstein–Wiener (4th order)	27.0	–
NARX (3rd order, wavelet)	24.5	–
NARX (3rd order, Tree partition)	19.3	–
NARX (3rd order, sigmoid network)	8.24	–
Standard GP-NARX	13.31	13.71
Var. Sparse GP-NARX ($M = 100$)	13.83	14.44
Reduced-rank GP-SSM (Svensson et al., 2016)	8.17	3.71
SISOG (Bijl et al., 2016)	7.12	NA
REVARB ($H = 2$)	6.04	3.04
Local S-REVARB ($H = 2$)	8.35	3.44
Global S-REVARB ($H = 2$)	7.46	3.32

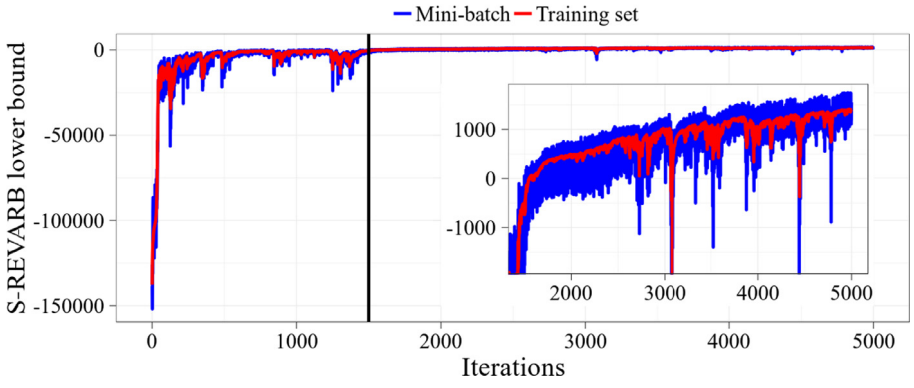
7.2. Initial example

We first tackle the task of modeling the nonlinear dynamics of a Magneto-Rheological (MR) Fluid Damper, an experiment described in Wang, Sano, Chen, and Huang (2009).[7] The damper is a semi-active control device to reduce vibrations on dynamical structures. The variable viscosity of the MR fluid is used to control the damping force. The *Damper* dataset consists of inputs related to the system velocity and outputs related to the achieved damping force for a given fluid viscosity. The same set-up presented in Svensson et al. (2016) is applied, which consists of using the first 2000 (out of the 3499) data samples for training and the remaining for test via free simulation. Such smaller dataset will enable better comparison with the batch version of the REVARB framework.

[7] Data available in the System Identification Toolbox for Mathworks Matlab (The MathWorks Inc., 2016).



(a) Local S-REVARB.



(b) Global S-REVARB.

Fig. 5. Convergence curves of the S-REVARB lower bound with $H = 2$ hidden layers, in both Local and Global variants, during the training step on the *Damper* dataset using the Adam stochastic gradient algorithm. The vertical **black** line indicates the instant where the kernel hyperparameters are unfixed and the smaller pictures are zoomed versions of the curves after such instant.

The experiments with the *Damper* dataset are performed with 3 RGP models containing $H = 2$ hidden layers and inference via standard batch REVARB, Local and Global S-REVARB algorithms. The models use orders $L = L_u = 3$ and $M = 100$ pseudo-inputs. Both stochastic variants use mini-batches of size $B = 200$, while the recognition models in the Global S-REVARB experiment have 100 hidden units. Standard REVARB was optimized with the BFGS algorithm, while the other two approaches followed Adam stochastic updates for 5000 iterations.

Results are presented in Table 3, along with the results for some more conventional system identification methods reported in The MathWorks Inc. (2016). We also include the recent result obtained by a reduced-rank GP-SSM, which applies a particle MCMC strategy to perform Bayesian inference, reported in Svensson et al. (2016), and the result obtained in Bijl et al. (2016) with the SISOG (System Identification using Sparse Online GP) algorithm. We can see that both S-REVARB algorithms were able to obtain results not too far away from the batch REVARB. The Global S-REVARB was better than most other models, with the exception of the RMSE obtained by the SISOG method. The local S-REVARB also presented a good result, with slightly worse RMSE than the reduced-rank GP-SSM, but better NLPD value. We note that the S-REVARB lower bound is looser than the collapsed REVARB bound, so it is somehow expected to obtain better results with the latter in scenarios with small and medium datasets, where it is feasible to use all training samples in the batch approach. A similar argument can be made towards the somehow good result obtained by the NARX sigmoid network, since it applies the Levenberg–Marquardt optimization algorithm, which although usually better than first order methods, does not scale well for larger datasets. Besides, such model also does not provide prediction intervals (at least not so easily as GP-based approaches).

Fig. 5 illustrates the convergence curves of the S-REVARB lower bound computed over the current mini-batch and over the entire training set using both stochastic inference strategies. The vertical black line indicates the instant where the kernel hyperparameters are unfixed and the smaller pictures are zoomed versions of the curves after such instant. We can see that the Local S-REVARB was able to obtain a higher value for the lower bound. This was the usual behavior in our experiments with smaller datasets, when several cycles (epochs) over the data are possible and the local unconstrained variational parameters seem to enable further optimization of the bound. However, we emphasize that such additional optimization does not affect free simulation performance with test data, as can be noticed from the better predictive results obtained by the Global S-REVARB (Table 3), despite the lower value obtained for the bound.

Moreover, Fig. 5 also reveals that the Local S-REVARB presents a smoother convergence, while the Global S-REVARB is wigglier, which in the latter case is related to the update of all the weights of the NN recognition models in every iteration. Although in both cases the bound value in the mini-batch is noisier than the bound computed for the whole training data, they follow similar behavior, which indicates that the stochastic updates are valid approximations.

7.3. Stochastic system identification with large datasets

We now perform experiments with two much larger system identification benchmarks. The *Silverbox* dataset was the subject

Table 4
Summary of results for the free simulation on test data after estimation from large dynamical datasets. Note that only the stochastic methods (RNNs and S-REVARB variants) used the entire training sets, since they can be trained via mini-batches. The other models were optimized in batch using the first $N = 5000$ training samples.

Silverbox	RMSE	NLPD
RNN (1 hidden layer)	2.107×10^{-3}	–
RNN (2 hidden layers)	3.369×10^{-3}	–
Var. Sparse GP-NARX ($N = 5000$)	1.676×10^{-3}	−4.204
REVARB ($H = 1, N = 5000$)	1.757×10^{-3}	−4.217
REVARB ($H = 2, N = 5000$)	3.562×10^{-3}	−3.997
Local S-REVARB ($H = 1$)	1.245×10^{-3}	−4.222
Local S-REVARB ($H = 2$)	8.285×10^{-3}	−3.321
Global S-REVARB ($H = 1$)[a]	$\mathbf{1.052 \times 10^{-3}}$	**−4.226**
Global S-REVARB ($H = 2$)[a]	1.4031×10^{-3}	−4.1436
Wiener–Hammerstein	RMSE	NLPD
RNN (1 hidden layer)	1.222×10^{-2}	–
RNN (2 hidden layers)	8.247×10^{-3}	–
Var. Sparse GP-NARX ($N = 5000$)	3.584×10^{-2}	−1.883
REVARB ($H = 1, N = 5000$)	2.037×10^{-2}	−2.406
REVARB ($H = 2, N = 5000$)	1.547×10^{-2}	−2.544
Local S-REVARB ($H = 1$)	1.295×10^{-2}	−2.609
Local S-REVARB ($H = 2$)	2.372×10^{-2}	−2.308
Global S-REVARB ($H = 1$)[b]	8.369×10^{-3}	−2.606
Global S-REVARB ($H = 2$)[b]	$\mathbf{5.664 \times 10^{-3}}$	**−2.643**

[a]Variational sparse simulation.
[b]Recognition-based simulation.

Table 5
Summary of p-values related to the application of the Wilcoxon signed rank nonparametric statistical test in the absolute errors obtained by the proposed models that presented the best results in Table 4 and the other models in the same table. All the reported values indicate that the null hypothesis can be rejected for a significance level of $p < 0.01$.

Silverbox Best model: Global S-REVARB ($H = 1$)	p-value
RNN (1 hidden layer)	2.036×10^{-192}
RNN (2 hidden layers)	0.000×10^{0}
Var. Sparse GP-NARX ($N = 5000$)	0.000×10^{0}
REVARB ($H = 1, N = 5000$)	0.000×10^{0}
REVARB ($H = 2, N = 5000$)	0.000×10^{0}
Local S-REVARB ($H = 1$)	5.306×10^{-3}
Local S-REVARB ($H = 2$)	0.000×10^{0}
Global S-REVARB ($H = 2$)	0.000×10^{0}
Wiener–Hammerstein Best model: Global S-REVARB ($H = 2$)	p-value
RNN (1 hidden layer)	0.000×10^{0}
RNN (2 hidden layers)	1.667×10^{-131}
Var. Sparse GP-NARX ($N = 5000$)	0.000×10^{0}
REVARB ($H = 1, N = 5000$)	0.000×10^{0}
REVARB ($H = 2, N = 5000$)	0.000×10^{0}
Local S-REVARB ($H = 1$)	0.000×10^{0}
Local S-REVARB ($H = 2$)	0.000×10^{0}
Global S-REVARB ($H = 1$)	0.000×10^{0}

of a special session organized at the IFAC Symposium on Nonlinear Control Systems (NOLCOS) in 2004 (Schoukens, Nemeth, Crama, Rolain, & Pintelon, 2003; Wigren & Schoukens, 2013).[8] Its data comes from an electrical circuit describing the behavior of a mass–spring–damper nonlinear dynamical system with feedback, where the linear contributions are dominant. The input is related to the force applied to the mass and the output represents the mass displacement. A total of 91,072 samples are used for training and 40,000 for testing.

The *Wiener–Hammerstein* benchmarking dataset (Hjalmarsson, Rojas, & Rivera, 2012; Schoukens, Suykens, & Ljung, 2009),[9] presented in the IFAC Symposium on System Identification in 2009 is comprised of data from an electronic nonlinear system consisting of a cascade of a linear dynamical block, a static nonlinear block and a final linear dynamical block. The training and test sets contain 95,000 and 84,000 samples, respectively. We have skipped the first 5000 constant samples from the original 100,000 training observations and the last 4000 zero value test samples from the original 88,000.

Both the original *Silverbox* and *Wiener–Hammerstein* datasets are almost absent of observation noise, with the *Silverbox* having negligible noise and the *Wiener–Hammerstein* presenting a very high Signal-to-Noise Ratio (SNR) of 70 dB. In order to evaluate our approaches with noisier data, we add zero-mean Gaussian noise to both training sets with variances computed to set their SNRs approximately equal to 20 dB.

In all experiments we used the orders $L = L_u = 10$ and $M = 50$ pseudo-inputs. The mini-batch size was fixed to $B = 1000$, which showed empirically to be a good balance between stable gradients and computational cost. For the Global S-REVARB we use networks with 100 hidden units for the recognition models of both the variational means and variances.

For comparison, we have included in the experiments the variational sparse GP-NARX and RGP models with the standard REVARB method. In those cases we used only the first 5000 training samples for the estimation step. We also experimented with the same RNN applied as a sequential recognition model in the Global S-REVARB used as a stand-alone model, with both 1 and 2 hidden layers and 100 hidden units per layer. Those are initialized and stochastically optimized similarly to the Global S-REVARB algorithm.

The summary of obtained RMSE and NLPD values are presented in Table 4, while p-values related to the application of the Wilcoxon signed rank nonparametric statistical test are reported in Table 5. The latter was performed for the models that presented the best RMSE values in Table 4 against the remaining models.

In the case of the *Silverbox* dataset, the 1-hidden layer versions of all the multilayer models performed better than their 2-hidden layer counterparts. This was somehow expected, since in this dataset the linear dynamics are dominant, as remarked in Marconato, Sjöberg, Suykens, and Schoukens (2012), and the inclusion of a second nonlinear recurrent layer proved to be more of a burden. The best result was obtained by the Global S-REVARB with $H = 1$, but the same model trained with the Local S-REVARB obtained the second best result.

Fig. 6 compares the absolute errors achieved by the Global S-REVARB solution with $H = 1$ and the 1-hidden layer RNN, where the former presents lower values in most areas. The final 5000 test samples are highlighted in the subfigure because they are related to inputs outside the range used to generate the training samples, which make them more difficult to predict. Still, the S-REVARB also presented smaller error values in that segment.

In the results for the *Wiener–Hammerstein* benchmark, also presented in Table 4, we had the opposite behavior. Most of the multilayer models with 2 hidden layers performed better than their shallow versions. The exception was the Local S-REVARB. This time the best solution was the Global S-REVARB with $H = 2$. Fig. 7 presents the absolute test errors obtained by that best solution and the RNN with two hidden layers, where we can see that the largest errors are associated with the RNN model.

The not so good results obtained by the Local S-REVARB in the experiments with the *Wiener–Hammerstein* dataset may be related

[8] Data available at http://www.it.uu.se/research/publications/reports/2013-006/SNLA80mVZipped.zip.

[9] Data available at http://www.ee.kth.se/~hjalmars/ifac_tc11_benchmarks/2009_wienerhammerstein/WienerHammerBenchMark.mat.

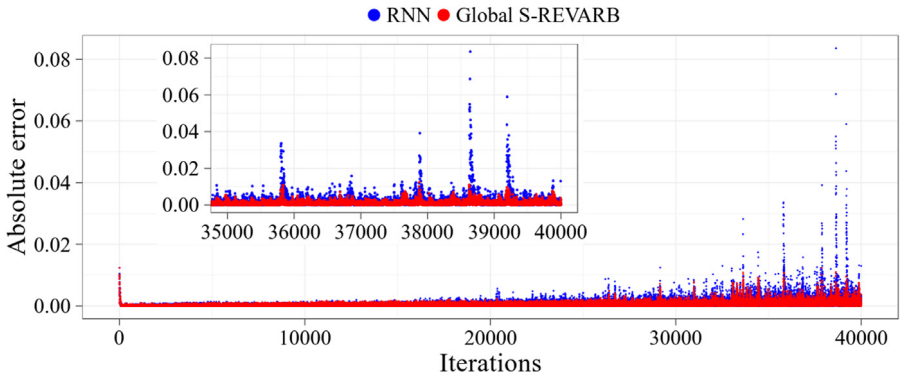


Fig. 6. Comparison between the absolute test errors obtained by the Global S-REVARB and the RNN on the *Silverbox* dataset, both with one hidden layer. The smaller picture is a zoomed version of the last 5000 test samples, the most difficult ones to predict correctly, since they were generated with inputs outside the range used during the training step.

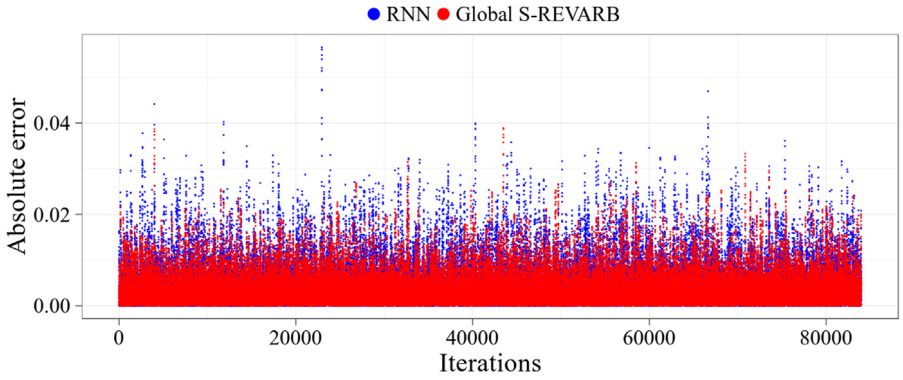


Fig. 7. Comparison between the absolute test errors obtained by the Global S-REVARB and the RNN on the *Wiener–Hammerstein* dataset, both with two hidden layers.

to the information provided by Table 6, which reports the number of adjustable parameters, hyperparameters and variational parameters present in the models that had access to the entire training set. As can be seen, the size of the Local S-REVARB implementation becomes considerably large with the increase of the number of training samples N, which may turn the optimization procedure more difficult, even though it is scalable in terms of computation and memory demands due to the mini-batch training. Moreover, since we have used 10,000 optimization iterations and mini-batches of $B = 1000$ size sampled from $N = 95{,}000$, each local variational parameter was stochastically update only 106 times. The good result obtained by the Local S-REVARB with one hidden for the *Silverbox* is possibly related to the simpler dynamics of that dataset, which did not require so many update steps from the initialization to get a reasonable model.

On the other hand, the size of the Global S-REVARB implementation scales (linearly) only with the number of hidden layers and do not increase with the amount of available data, since there is no local variational parameters and the quantity of weights of the sequential recognition models does not grow with N. The great difference in size between the two S-REVARB methods, where the Local S-REVARB approach presents up to 25 times more adjustable parameters than its Global equivalent in Table 6, may justify the better performance of the latter in the reported results.

Table 6
Comparison of the number of adjustable parameters (RNNs) or hyperparameters and variational parameters (S-REVARB variants) in the experiments with the *Wiener–Hammerstein* benchmark ($N = 95{,}000$).

	Size
RNN (1 hidden layer)	2 201
RNN (2 hidden layers)	4 402
Local S-REVARB ($H = 1$)	194,206
Local S-REVARB ($H = 2$)	386,574
Global S-REVARB ($H = 1$)	8 608
Global S-REVARB ($H = 2$)	15,378

8. Conclusions

In this work we have considered the challenge of learning dynamical models from large sequential datasets. The scalability issues of GP-based models, which even in their sparse versions still scale at least linearly with the number of training samples, were tackled by following stochastic optimization procedures.

More precisely, inspired by recent advances on stochastic variational inference, we developed a non-collapsed factorizable modification of the original batch REVARB lower bound to the model marginal log-likelihood presented in Mattos et al. (2016), resulting in the S-REVARB framework, which aims for better scalable inference with the RGP model.

From such formulation, we described two stochastic learning algorithms, the Local S-REVARB and the Global S-REVARB. The former more directly applies the standard SVI ideas to the new factorized bound, keeping all the local variational parameters. The latter incorporates sequential recognition models in order to include NN-based constraints to the S-REVARB inference framework, which no longer contains local parameters and is even further scalable. Both algorithms enable optimization using noisy mini-batch updates via off-the-shelf stochastic gradient ascent strategies, such as the ADAM optimizer. We note that more elaborate adaptive learning rate strategies introduced for other non-GP SVI algorithms, such as the ones presented in Ranganath, Wang, David, and Xing (2013) and Li and Ouyang (2016), were not explored in this work and is left for future investigations.

We evaluated the proposed solutions in the task of system identification from datasets with up to 95,000 training points, much more than which is computationally feasible with the original REVARB approach. The obtained results indicate that the stochastic variational framework is a viable scalable alternative to the REVARB batch approach, especially the Global S-REVARB variant, which avoids the increase of its implementation size with the number of training samples and has presented the best overall results.

In summary, this paper greatly extends the possible scenarios where GP-based dynamical modeling can be applied, aiming for scalability without sacrificing the expressiveness of the hierarchical recurrent structure of RGP models. An important open issue lies in the task of learning from sequential data contaminated by outliers. That task was successfully tackled with the RGP model recently (Mattos et al., 2017), but in a batch context. The problem of dealing with outliers in a stochastic or online setting was covered by Vuković and Miljković (2015) with neural networks. Interestingly, both previous works use heavy-tailed likelihoods and variational approximations. How to adapt the proposed S-REVARB framework for outlier-robust learning is left for future investigations.

Acknowledgments

The authors acknowledge the financial support of the following Brazilian funding agencies: FUNCAP, Brazil and CAPES, Brazil for the PhD scholarships granted to the first author, and CNPq, Brazil for the grant 309451/2015-9 awarded to the second author. The authors also would like to thank Zhenwen Dai and Andreas Damianou for the valuable discussions on the initial ideas for this work.

Appendix A. Full S-REVARB lower bound

After some manipulation of the bound presented in Eq. (25), the final uncollapsed S-REVARB lower bound is given by Eq. (A.1). Note that in order to obtain the version used during mini-batch stochastic optimization of Eq. (30), we need to scale the large summation by $\frac{N-L}{B}$ and compute it only for the sampled indexes.

$$
\begin{aligned}
\log p(\boldsymbol{y}) \geq \sum_{i=L+1}^{N} \Bigg\{ & -\frac{1}{2}\log 2\pi\sigma_{H+1}^2 \\
& - \frac{1}{2\sigma_{H+1}^2}\left(\Psi_0^{i(H+1)} + y_i^2 - \operatorname{Tr}\left(\left(\boldsymbol{K}_z^{(H+1)}\right)^{-1}\boldsymbol{\Psi}_2^{i(H+1)}\right)\right) \\
& + \frac{1}{\sigma_{H+1}^2} y_i \left(\boldsymbol{\Psi}_1^{i(H+1)}\right)^{\top}\left(\boldsymbol{K}_z^{(H+1)}\right)^{-1}\boldsymbol{m}^{(H+1)} \\
& - \frac{1}{2\sigma_{H+1}^2}\operatorname{Tr}\left(\left(\boldsymbol{S}^{(H+1)} + \boldsymbol{m}^{(H+1)}\left(\boldsymbol{m}^{(H+1)}\right)^{\top}\right)\right. \\
& \left.\left(\boldsymbol{K}_z^{(H+1)}\right)^{-1}\boldsymbol{\Psi}_2^{i(H+1)}\left(\boldsymbol{K}_z^{(H+1)}\right)^{-1}\right) \\
& + \sum_{h=1}^{H}\Bigg[-\frac{1}{2}\log 2\pi\sigma_h^2 \\
& -\frac{1}{2\sigma_h^2}\left(\Psi_0^{i(h)} + \left(\mu_i^{(h)}\right)^2 + \lambda_i - \operatorname{Tr}\left(\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{\Psi}_2^{i(h)}\right)\right) \\
& + \frac{1}{\sigma_h^2}\mu_i^{(h)}\left(\boldsymbol{\Psi}_1^{i(h)}\right)^{\top}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{m}^{(h)} \\
& - \frac{1}{2\sigma_h^2}\operatorname{Tr}\left(\left(\boldsymbol{S}^{(h)} + \boldsymbol{m}^{(h)}\left(\boldsymbol{m}^{(h)}\right)^{\top}\right)\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{\Psi}_2^{i(h)}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\right) \\
& - \int_{\boldsymbol{x}} q\left(x_i^{(h)}\right)\log q\left(x_i^{(h)}\right)\Bigg]\Bigg\} \\
& - \frac{1}{2}\sum_{h=1}^{H+1}\Big[\operatorname{Tr}\left(\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{S}^{(h)}\right) + \left(\boldsymbol{m}^{(h)}\right)^{\top}\left(\boldsymbol{K}_z^{(h)}\right)^{-1}\boldsymbol{m}^{(h)} \\
& -M + \log\left|\boldsymbol{K}_z^{(h)}\right| - \log\left|\boldsymbol{S}^{(h)}\right|\Big] \\
& + \sum_{i=1}^{L}\sum_{h=1}^{H}\Bigg[\int_{\boldsymbol{x}} q\left(x_i^{(h)}\right)\log p\left(x_i^{(h)}\right) \\
& - \int_{\boldsymbol{x}} q\left(x_i^{(h)}\right)\log q\left(x_i^{(h)}\right)\Bigg].
\end{aligned}
\tag{A.1}
$$

Appendix B. Variational lower bound statistics

Here we detail the statistics defined in Eqs. (27)–(29), which are related to the ones presented in the original paper on Bayesian GP-LVM (Titsias & Lawrence, 2010) and also used, although in their batch form, in Mattos et al. (2016) for the standard REVARB method.

First, we define the notation of the following variational distribution based on Eq. (17):

$$
q\left(\hat{\boldsymbol{x}}_i^{(h)}\right) = \mathcal{N}\left(\hat{\boldsymbol{x}}_i^{(h)} \middle| \boldsymbol{\mu}_i^{(h)}, \boldsymbol{\Sigma}_i^{(h)}\right),
$$

where $\hat{\boldsymbol{x}}_i^{(h)} \in \mathbb{R}^{D_h}$, $\boldsymbol{\mu}_i^{(h)} \in \mathbb{R}^{D_h}$, $\boldsymbol{\Sigma}_i^{(h)} \in \mathbb{R}^{D_h \times D_h}$ and

$$
D_h = \begin{cases} L + L_u, & \text{if } h = 1, \\ 2L, & \text{if } 1 < h \leq H, \\ L, & \text{if } h = H+1. \end{cases}
$$

The moments themselves are given by

$$
\boldsymbol{\mu}_i^{(h)} = \begin{cases} \left[\bar{\boldsymbol{\mu}}_{i-1}^{(1)}, \bar{\boldsymbol{u}}_{i-1}\right], & \text{if } h = 1, \\ \left[\bar{\boldsymbol{\mu}}_{i-1}^{(h)}, \bar{\boldsymbol{\mu}}_i^{(h-1)}\right], & \text{if } 1 < h \leq H, \\ \bar{\boldsymbol{\mu}}_i^{(H)}, & \text{if } h = H+1, \end{cases}
$$

$$
\boldsymbol{\Sigma}_i^{(h)} = \begin{cases} \operatorname{diag}\left(\left[\bar{\boldsymbol{\lambda}}_{i-1}^{(1)}, \boldsymbol{0}_{L_u}\right]\right), & \text{if } h = 1, \\ \operatorname{diag}\left(\left[\bar{\boldsymbol{\lambda}}_{i-1}^{(h)}, \bar{\boldsymbol{\lambda}}_i^{(h-1)}\right]\right), & \text{if } 1 < h \leq H, \\ \operatorname{diag}\left(\bar{\boldsymbol{\lambda}}_i^{(H)}\right), & \text{if } h = H+1, \end{cases}
$$

where $\bar{\boldsymbol{\mu}}_{i-1}^{(h)} = \left[\mu_{i-1}^{(h)}, \ldots, \mu_{i-L}^{(h)}\right]$, $\bar{\boldsymbol{\lambda}}_{i-1}^{(h)} = \left[\lambda_{i-1}^{(h)}, \ldots, \lambda_{i-L}^{(h)}\right]$, $\bar{\boldsymbol{u}}_{i-1} = \left[u_{i-1}, \ldots, u_{i-L_u}\right]$, and the function $\operatorname{diag}(\cdot)$ builds a diagonal matrix from its argument. Note that $\boldsymbol{0}_{L_u}$ is a vector of L_u zeros, related to the variances of the external inputs, which are actually considered deterministic. Conveniently, if we have the uncertainty information about those variables, e.g., if they come from a Bayesian controller, we can simply replace this vector by the vector of known variances, given that they are (at least approximately) Gaussian distributed, without changing the framework. If that approach is pursued it is equivalent to approximately marginalize the external inputs u_i, similar to the marginalization of the dynamical latent variables $x_i^{(1)}$ of the first layer.

We can now detail the statistics in Eqs. (27)–(29). We will drop the indexes h that refer to the layer number to reduce clutter in

the notation, but we emphasize that the computations are separately performed for each layer. Below we list the expressions considering all the training samples, but in practice the statistics are calculated only from the variables related to the samples in a given mini-batch, i.e., $\forall i \in \mathcal{B}$. We consider that the exponentiated quadratic covariance function (Eq. (5)) was chosen.

The statistic $\Psi_0 = \mathrm{Tr}\left(\left\langle \boldsymbol{K}_f \right\rangle_{q(\boldsymbol{x})}\right) \in \mathbb{R}$ is given by

$$\Psi_0 = \sum_{i=1}^{N-L} \int_{\hat{\boldsymbol{x}}_i} k\left(\hat{\boldsymbol{x}}_i, \hat{\boldsymbol{x}}_i\right) \mathcal{N}\left(\hat{\boldsymbol{x}}_i \middle| \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i\right)$$

$$\Psi_0 = \sum_{i=1}^{N-L} \sigma_f^2.$$

The computation of each element of $\boldsymbol{\Psi}_1 = \left\langle \boldsymbol{K}_{fz} \right\rangle_{q(\boldsymbol{x})} \in \mathbb{R}^{(N-L)\times M}$ is performed as follows:

$$[\boldsymbol{\Psi}_1]_{ij} = \int_{\hat{\boldsymbol{x}}_i} k\left(\hat{\boldsymbol{x}}_i, \boldsymbol{\zeta}_j\right) \mathcal{N}\left(\hat{\boldsymbol{x}}_i \middle| \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i\right)$$

$$[\boldsymbol{\Psi}_1]_{ij} = \sigma_f^2 \prod_{d=1}^{D} \frac{\exp\left(-\frac{1}{2}\frac{w_d(x_{id}-\zeta_{jd})^2}{w_d[\boldsymbol{\Sigma}_i]_{dd}+1}\right)}{(w_d[\boldsymbol{\Sigma}_i]_{dd}+1)^{\frac{1}{2}}}.$$

The statistic $\boldsymbol{\Psi}_2 = \left\langle \left(\boldsymbol{K}_{fz}\right)^\top \boldsymbol{K}_{fz} \right\rangle_{q(\boldsymbol{x})} \in \mathbb{R}^{M\times M}$ is given by $\boldsymbol{\Psi}_2 = \sum_{i=1}^{N-L} \boldsymbol{\Psi}_2^i$, where each element of the matrices $\boldsymbol{\Psi}_2^i \in \mathbb{R}^{M\times M}$ is given by

$$\left[\boldsymbol{\Psi}_2^i\right]_{jj'} = \int_{\hat{\boldsymbol{x}}_i} k\left(\hat{\boldsymbol{x}}_i, \boldsymbol{\zeta}_j\right) k\left(\hat{\boldsymbol{x}}_i, \boldsymbol{\zeta}_{j'}\right) \mathcal{N}\left(\hat{\boldsymbol{x}}_i \middle| \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i\right)$$

$$\left[\boldsymbol{\Psi}_2^i\right]_{jj'} = \sigma_f^4 \prod_{d=1}^{D} \frac{\exp\left(-\frac{w_d(\zeta_{jd}-\zeta_{j'd})^2}{4} - \frac{w_d(x_{id}-\tilde{\zeta}_d)^2}{2w_d[\boldsymbol{\Sigma}_i]_{dd}+1}\right)}{(2w_d[\boldsymbol{\Sigma}_i]_{dd}+1)^{\frac{1}{2}}},$$

where $\tilde{\zeta}_d = \frac{(\zeta_{jd}+\zeta_{j'd})}{2}$.

References

Al-Shedivat, M., Wilson, A. G., Saatchi, Y., Hu, Z., & Xing, E. P. (2017). Learning scalable deep kernels with recurrent structure. *Journal of Machine Learning Research (JMLR)*, *18*(1), 2850–2886.

Amari, S. I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, *10*(2), 251–276.

Ambikasaran, S., Foreman-Mackey, D., Greengard, L., Hogg, D. W., & O'Neil, M. (2016). Fast direct methods for Gaussian processes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *38*(2), 252–265.

Ažman, K., & Kocijan, J. (2011). Dynamical systems identification using Gaussian process models with incorporated local models. *Engineering Applications of Artificial Intelligence*, *24*(2), 398–408.

Barber, D. (2012). *Bayesian reasoning and machine learning*. Cambridge, UK: Cambridge University Press.

Bijl, H., Schön, T. B., van Wingerden, J. W., & Verhaegen, M. (2016). Online sparse Gaussian process training with input noise, arXiv preprint arXiv:1601.08068, URL https://arxiv.org/abs/1601.08068.

Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational inference: A review for statisticians. *Journal of the American Statistical Association*, *112*(518), 859–877.

Bottou, L. (1999). Online learning and stochastic approximations. In D. Saad (Ed.), *On-line learning in neural networks* (pp. 9–42). Cambridge University Press.

Bottou, L. (2004). Stochastic learning. In *Advanced lectures on machine learning* (pp. 146–168). New York, NY, USA: Springer.

Brahim-Belhouari, S., & Bermak, A. (2004). Gaussian process for nonstationary time series prediction. *Computational Statistics & Data Analysis*, *47*(4), 705–712.

Bui, T. D., Hernández-Lobato, D., Li, Y., Hernández-Lobato, J. M., & Turner, R. E. (2016). Deep Gaussian processes for regression using approximate expectation propagation. In *Proceedings of The 33rd international conference on machine learning*. New York City, NY, USA: JMLR.org.

Bui, T. D., & Turner, R. E. (2014). Tree-structured Gaussian process approximations. In *Advances in Neural Information Processing Systems 27* (pp. 2213–2221). Montréal, Canada: NIPS Foundation.

Bui, T. D., & Turner, R. E. (2015). Stochastic variational inference for Gaussian process latent variable models using back constraints. In *NIPS workshop on black box learning and inference*. Montréal, Canada: NIPS Foundation.

Carli, F. P., Chiuso, A., & Pillonetto, G. (2012). Efficient algorithms for large scale linear system identification using stable spline estimators. In *Proceedings of the 16th IFAC symposium on system identification, Vol. 45* (pp. 119–124). Brussels, Belgium: Elsevier.

Cheng, Y., Wang, Y., Camps, O., & Sznaier, M. (2015). The interplay between big data and sparsity in systems identification: some lessons from machine learning. In *Proceedings of the 17th IFAC symposium on system identification, Vol. 48* (pp. 1285–1292). Beijing, China: Elsevier.

Dai, Z., Damianou, A., González, J., & Lawrence, N. (2016). Variational auto-encoded deep Gaussian processes. In *International conference on learning representations*. San Juan Puerto Rico: ICLR, URL https://arxiv.org/pdf/1511.06455.

Dai, Z., Damianou, A., Hensman, J., & Lawrence, N. (2014). Gaussian process models with parallelization and GPU acceleration, arXiv preprint arXiv:1410.4984, URL https://arxiv.org/pdf/1410.4984.

Damianou, A. (2015). *Deep Gaussian processes and variational propagation of uncertainty* (Ph.D. thesis), University of Sheffield.

Deisenroth, M. P., & Ng, J. W. (2015). Distributed Gaussian processes. In *Proceedings of the 32nd international conference on machine learning* (pp. 1481–1490). Lille, France: JMLR.org.

Deisenroth, M. P., Turner, R. D., Huber, M. F., Hanebeck, U. D., & Rasmussen, C. E. (2012). Robust filtering and smoothing with Gaussian processes. *IEEE Transactions on Automatic Control*, *57*(7), 1865–1871.

Dezfouli, A., & Bonilla, E. V. (2015). Scalable inference for Gaussian process models with black-box likelihoods. In *Advances in neural information processing systems 28* (pp. 1414–1422). Montréal, Canada: NIPS Foundation.

Eleftheriadis, S., Nicholson, T., Deisenroth, M., & Hensman, J. (2017). Identification of Gaussian process state space models. In *Advances in neural information processing systems* (pp. 5309–5319).

Fletcher, R. (2013). *Practical methods of optimization*. Hoboken, NJ, USA: John Wiley & Sons.

Frigola-Alcade, R., Chen, Y., & Rasmussen, C. (2014). Variational Gaussian process state-space models. In *Advances in neural information processing systems 27* (pp. 3680–3688). Cambridge, MA, USA: MIT Press.

Frigola-Alcade, R., Lindsten, F., Schön, T. B., & Rasmussen, C. E. (2013). Bayesian inference and learning in Gaussian process state-space models with particle MCMC. In *Advances in Neural Information Processing Systems 26* (pp. 3156–3164). Lake Tahoe, Nevada, USA: NIPS Foundation.

Frigola-Alcade, R., & Rasmussen, C. E. (2013). Integrated pre-processing for Bayesian nonlinear system identification with Gaussian processes. In *52nd IEEE conference on decision and control* (pp. 5371–5376). Firenze, Italy: IEEE.

Gal, Y., & Ghahramani, Z. (2016). A theoretically grounded application of dropout in recurrent neural networks. In *Advances in neural information processing systems* (pp. 1019–1027).

Gal, Y., van der Wilk, M., & Rasmussen, C. E. (2014). Distributed variational inference in sparse Gaussian process regression and latent variable models. In *Advances in neural information processing systems 27* (pp. 3257–3265). Montréal, Canada: NIPS Foundation.

Gelman, A., Vehtari, A., Jylänki, P., Robert, C., Chopin, N., & Cunningham, J. P. (2014). Expectation propagation as a way of life, arXiv preprint arXiv:1412.4869, URL https://arxiv.org/pdf/1412.4869.

Girard, A., Rasmussen, C. E., Quiñonero-Candela, J., & Murray-Smith, R. (2002). Gaussian process priors with uncertain inputs - application to multiple-step ahead time series forecasting. In *Advances in neural information processing systems 15*. Vancouver, Canada: MIT Press.

Girard, A., Rasmussen, C., Quiñonero-Candela, J., & Murray-Smith, R. (2003). Multiple-step ahead prediction for non linear dynamic systems: a Gaussian process treatment with propagation of the uncertainty. In *Advances in neural information processing systems 16* (pp. 529–536). Cambridge, MA, USA: MIT Press.

Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the 14th international conference on artificial intelligence and statistics, Vol. 15* (pp. 315–323). Ft. Lauderdale, Florida, USA: JMLR.org.

Green, P., Cross, E., & Worden, K. (2015). Bayesian system identification of dynamical systems using highly informative training data. *Mechanical Systems and Signal Processing*, *56*, 109–122.

Green, P., & Maskell, S. (2017). Estimating the parameters of dynamical systems from big data using sequential Monte Carlo samplers. *Mechanical Systems and Signal Processing*, *93*, 379–396.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision* (pp. 1026–1034). Santiago, Chile: IEEE.

Hensman, J., Damianou, A., & Lawrence, N. (2014). Opening the way for deep Gaussian processes on massive data. In *International conference on artificial intelligence and statistics*. Late Breaking Poster.

Hensman, J., Durrande, N., & Solin, A. (2016). Variational fourier features for Gaussian processes, arXiv preprint arXiv:1611.06740, URL https://arxiv.org/abs/1611.06740.

Hensman, J., Fusi, N., & Lawrence, N. D. (2013). Gaussian processes for big data. In *29th conference on uncertainty in artificial intelligence* (pp. 282–290). Bellevue, Washington, USA: AUAI Press.

Hensman, J., Matthews, A. G. d. G., & Ghahramani, Z. (2015). Scalable variational Gaussian process classification. In *Proceedings of the 18th international conference on artificial intelligence and statistics*. San Diego, California, USA: JMLR.org.

Hjalmarsson, H., Rojas, C. R., & Rivera, D. E. (2012). System identification: a Wiener-Hammerstein benchmark. *Control Engineering Practice*, *20*(11), 1095–1096.

Hoang, T. N., Hoang, Q. M., & Low, B. K. H. (2015). A unifying framework of anytime sparse Gaussian process regression models with stochastic variational inference for big data. In *Proceedings of the 32nd international conference on machine learning* (pp. 569–578). Lille, France: JMLR.org.

Hoang, T. N., Hoang, Q. M., & Low, B. K. H. (2016). A distributed variational inference framework for unifying parallel sparse Gaussian process regression models. In *Proceedings of the 33rd international conference on machine learning* (pp. 382–391). New York City, NY, USA: JMLR.org.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Hoffman, M. D., Blei, D. M., Wang, C., & Paisley, J. W. (2013). Stochastic variational inference. *Journal of Machine Learning Research (JMLR)*, *14*(1), 1303–1347.

Isermann, R., & Münchhof, M. (2011). *Identification of dynamic systems*. Berlin, Alemanha: Springerverlag Berlin Heidelberg.

Jordan, M. I., Ghahramani, Z., Jaakkola, T. S., & Saul, L. K. (1999). An introduction to variational methods for graphical models. *Machine Learning*, *37*(2), 183–233.

Kantas, N., Doucet, A., Singh, S. S., Maciejowski, J., Chopin, N., et al. (2015). On particle methods for parameter estimation in state-space models. *Statistical Science*, *30*(3), 328–351.

Kim, Y., Mallick, R., Bhowmick, S., & Chen, B. -L. (2013). Nonlinear system identification of large-scale smart pavement systems. *Expert Systems with Applications*, *40*(9), 3551–3560.

Kingma, D., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International conference on learning representations*. San Diego, California, USA: ICLR.

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes.

Kocijan, J. (2016). *Modelling and control of dynamic systems using Gaussian process models*. New York, NY, USA: Springer.

Kocijan, J., Girard, A., Banko, B., & Murray-Smith, R. (2005). Dynamic systems identification with Gaussian processes. *Mathematical and Computer Modelling of Dynamical Systems*, *11*(4), 411–424.

Lawrence, N. D. (2004). Gaussian process latent variable models for visualisation of high dimensional data. In *Advances in neural information processing systems 17* (pp. 329–336). Vancouver and Whistler, British Columbia, Canada: MIT Press.

Li, X. M., & Ouyang, J. H. (2016). Tuning the learning rate for stochastic variational inference. *Journal of Computer Science and Technology*, *31*(2), 428.

Liu, H., Ong, Y. S., Shen, X., & Cai, J. (2018). When Gaussian process meets big data: A review of scalable GPs, arXiv preprint arXiv:1807.01065.

Marconato, A., Sjöberg, J., Suykens, J., & Schoukens, J. (2012). Identification of the silverbox benchmark using nonlinear state-space models. *45*, In *Proceedings of the 16th IFAC symposium on system identification* (pp. 632–637). Brussels, Belgium: Elsevier.

Mattos, C. L. C., Dai, Z., Damianou, A., Barreto, G. A., & Lawrence, N. D. (2017). Deep recurrent Gaussian processes for outlier-robust system identification. *Journal of Process Control*, *60*, 82–94.

Mattos, C. L. C., Dai, Z., Damianou, A., Forth, J., Barreto, G. A., & Lawrence, N. D. (2016). Recurrent Gaussian processes. In *International conference on learning representations*. San Juan Puerto Rico: ICLR, URL https://arxiv.org/pdf/1511.06644.

Mattos, C. L. C., Santos, J. D. A., & Barreto, G. A. (2015). An empirical evaluation of robust Gaussian process models for system identification. In *Intelligent data engineering and automated learning* (pp. 172–180). Wroclaw, Poland: Springer.

Minka, T. P. (2001). Expectation propagation for approximate Bayesian inference. In *Proceedings of the 17th conference on uncertainty in artificial intelligence* (pp. 362–369). Seattle, WA, USA: Morgan Kaufmann.

Murray-Smith, R., Johansen, T. A., & Shorten, R. (1999). On transient dynamics, off-equilibrium behaviour and identification in blended multiple model structures. In *European control conference*. Karlsruhe, Germany: Springer.

Neal, R. M. (1997). *Monte Carlo implementation of Gaussian process models for Bayesian regression and classification, Tech. rep.*. Toronto, Canada: University of Toronto, Dept. of Statistics.

Nelles, O. (2013). *Nonlinear system identification: from classical approaches to neural networks and fuzzy models*. Berlin, Germany: Springer Science & Business Media.

Nguyen, T., & Bonilla, E. (2014). Fast allocation of Gaussian process experts. In *Proceedings of the 31st international conference on machine learning* (pp. 145–153). Beijing, China: JMLR.org.

Nickson, T., Gunter, T., Lloyd, C., Osborne, M. A., & Roberts, S. (2015). Blitzkriging: Kronecker-structured Stochastic Gaussian processes, arXiv preprint arXiv:1510.07965, URL https://arxiv.org/pdf/1510.07965.

Quiñonero-Candela, J., & Rasmussen, C. E. (2005). A unifying view of sparse approximate Gaussian process regression. *The Journal of Machine Learning Research (JMLR)*, *6*, 1939–1959.

(2017). *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing, URL https://www.R-project.org/.

Raissi, M. (2017). Parametric Gaussian process regression for big data, arXiv preprint arXiv:1704.03144, URL https://arxiv.org/pdf/1704.03144.

Ranganath, R., Wang, C., David, B., & Xing, E. (2013). An adaptive learning rate for stochastic variational inference. In *Proceedings of the 30th international conference on machine learning* (pp. 298–306). Atlanta, USA: JMLR.org.

Rasmussen, C., & Williams, C. (2006). *Gaussian processes for machine learning* (1st ed.). Cambridge, MA, USA: MIT Press.

Reece, S., & Roberts, S. (2010). An introduction to Gaussian processes for the Kalman filter expert. In *13th conference on information fusion* (pp. 1–9). Edinburgh, UK: IEEE.

Rezende, D. J., Mohamed, S., & Wierstra, D. (2014). Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st international conference on machine learning*. Beijing, China: JMLR.org.

Robbins, H., & Monro, S. (1951). A stochastic approximation method. *The Annals of Mathematical Statistics*, 400–407.

Rottmann, A., & Burgard, W. (2010). Learning non-stationary system dynamics online using Gaussian processes. In *Lecture notes in computer science*: *Vol. 6373, Joint pattern recognition symposium* (pp. 192–201). Springer.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1985). *Learning internal representations by error propagation, Tech. rep.* (pp. 318–362). Cambridge, Massachusetts, USA: MIT Press, DTIC Document.

Salimbeni, H., & Deisenroth, M. (2017). Doubly stochastic variational inference for deep Gaussian processes, arXiv preprint arXiv:1705.08933, URL https://arxiv.org/abs/1705.08933.

Santos, J. D. A., & Barreto, G. A. (2017). A regularized estimation framework for online sparse LSSVR models. *Neurocomputing*, *238*, 114–125.

Särkkä, Solin, A., & Hartikainen, J. (2013). Spatiotemporal learning via infinite-dimensional Bayesian filtering and smoothing: A look at Gaussian process regression through Kalman filtering. *IEEE Signal Processing Magazine*, *30*(4), 51–61.

Schön, T. B., Lindsten, F., Dahlin, J., Wågberg, J., Naesseth, C. A., Svensson, A., et al. (2015). Sequential Monte Carlo methods for system identification. *48*, In *Proceedings of the 17th IFAC symposium on system identification* (pp. 775–786). Beijing, China: Elsevier.

Schoukens, J., Nemeth, J. G., Crama, P., Rolain, Y., & Pintelon, R. (2003). Fast approximate identification of nonlinear systems. *Automatica*, *39*(7), 1267–1274.

Schoukens, J., Suykens, J., & Ljung, L. (2009). Wiener-Hammerstein benchmark. In *Proceedings of the 15th IFAC symposium on system identification*. Saint-Malo, France: Elsevier.

Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in neural information processing systems 25* (pp. 2951–2959). Lake Tahoe, Nevada, USA: NIPS Foundation.

Solak, E., Murray-Smith, R., Leithead, W. E., Leith, D. J., & Rasmussen, C. E. (2003). Derivative observations in Gaussian process models of dynamic systems. In *Advances in neural information processing systems 16* (p. 16).

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research (JMLR)*, *15*(1), 1929–1958.

Svensson, A., Solin, A., Särkkä, S., & Schön, T. B. (2016). Computationally efficient Bayesian learning of Gaussian process state space models. In *Proceedings of the 19th international conference on artificial intelligence and statistics* (pp. 213–221). Cadiz, Spain: JMLR.org.

Talbi, E. -G. (2009). *Metaheuristics: from design to implementation, Vol. 74*. Hoboken, NJ, USA: John Wiley & Sons.

The MathWorks Inc., Nonlinear Modeling of a Magneto-Rheological Fluid Damper, URL http://www.mathworks.com/help/ident/examples/nonlinear-modeling-of-a-magneto-rheological-fluid-damper.html.

Titsias, M. K. (2009a). *Variational model selection for sparse Gaussian process regression, Tech. rep., Manchester, UK, Technical report*. School of Computer Science, University of Manchester.

Titsias, M. K. (2009b). Variational learning of inducing variables in sparse Gaussian processes. In *Proceedings of the 12th international conference on artificial intelligence and statistics* (pp. 567–574). Clearwater Beach, FL, USA: JMLR.org.

Titsias, M. K., & Lawrence, N. D. (2010). Bayesian Gaussian process latent variable model. In *Proceedings of the 13th international conference on artificial intelligence and statistics* (pp. 844–851). Sardinia, Italy: JMLR.org.

Turner, R. D., Deisenroth, M. P., & Rasmussen, C. E. (2010). State-space inference and learning with Gaussian processes. In *Proceedings of the 13th international conference on artificial intelligence and statistics* (pp. 868–875). Sardinia, Italy: JMLR.org.

Vuković, N., & Miljković, Z. (2015). Robust sequential learning of feedforward neural networks in the presence of heavy-tailed noise. *Neural Networks*, *63*, 31–47.

Wainwright, M. J., Jordan, M. I., et al. (2008). Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, *1*(1–2), 1–305.

Wang, J., Sano, A., Chen, T., & Huang, B. (2009). Identification of Hammerstein systems without explicit parameterisation of non-linearity. *International Journal of Control*, *82*(5), 937–952.

Wigren, T., & Schoukens, J. (2013). *Data for benchmarking in nonlinear system identification, Tech. rep., Uppsala, Sweden, Technical report*. Uppsala University, Dept. of Information Technology.

Williams, R. J., & Zipser, D. (1995). Gradient-based learning algorithms for recurrent networks and their computational complexity. *Back-Propagation: Theory, Architectures and Applications*, 433–486.

Wilson, A. G., Dann, C., & Nickisch, H. (2015). Thoughts on massively scalable Gaussian processes, arXiv preprint arXiv:1511.01870, URL https://arxiv.org/pdf/1511.01870.

Wilson, A. G., Hu, Z., Salakhutdinov, R., & Xing, E. P. (2016a). Deep kernel learning. In *Proceedings of the 19th international conference on artificial intelligence and statistics* (pp. 370–378). Cadiz, Spain: JMLR.org.

Wilson, A. G., Hu, Z., Salakhutdinov, R. R., & Xing, E. P. (2016b). Stochastic variational deep kernel learning. In *Advances in neural information processing systems 29* (pp. 2586–2594). Barcelona, Spain: NIPS Foundation.

Wilson, A., & Nickisch, H. (2015). Kernel interpolation for scalable structured Gaussian processes (KISS-GP). In *Proceedings of the 32nd international conference on machine learning* (pp. 1775–1784). Lille, France: JMLR.org.